ECE 590
Neural-Network-Based Large Language Models

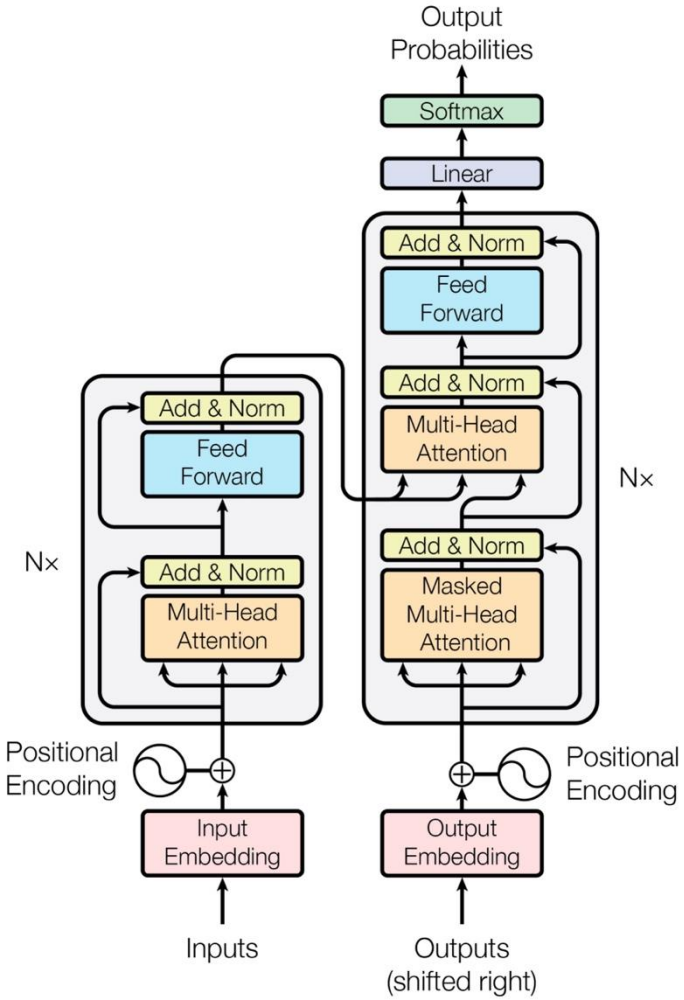
Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 12
02/17/2026

Model Simplification

- The original LLM used two “stacks”, one for the input and one for the output



Attention is all you need

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

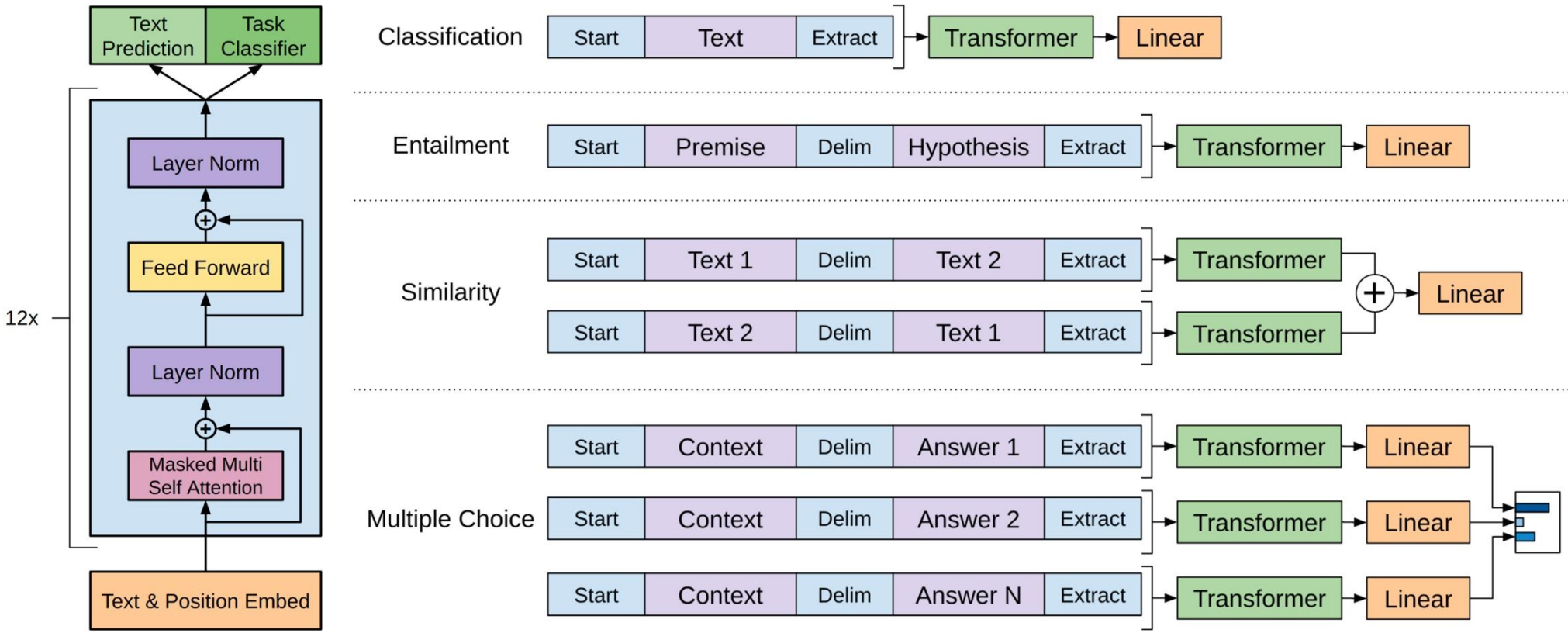
The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configuration. The best …

Save Cite Cited by 230906 Related articles

Model Simplification

- ❑ The original LLM used two “stacks”, one for the input and one for the output
- ❑ Use self-attention on each, as well as cross attention between output and input
- ❑ This design was constituted for historical reasons (*e.g.*, machine translation)
- ❑ LLM design has simplified significantly, employ a single stack of attention layers

GPT-1



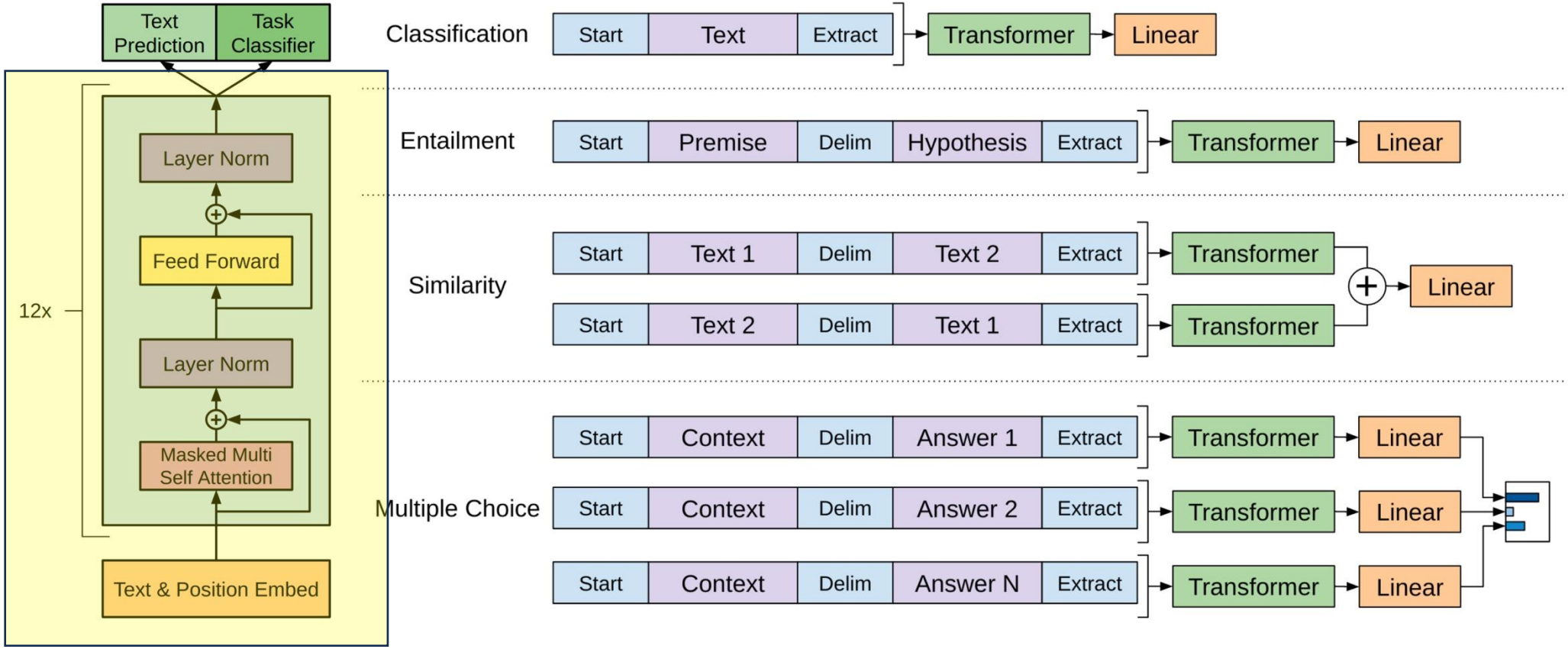
[PDF] Improving language understanding by generative pre-training

A Radford, K Narasimhan, T Salimans, I Sutskever - mikecaptain.com

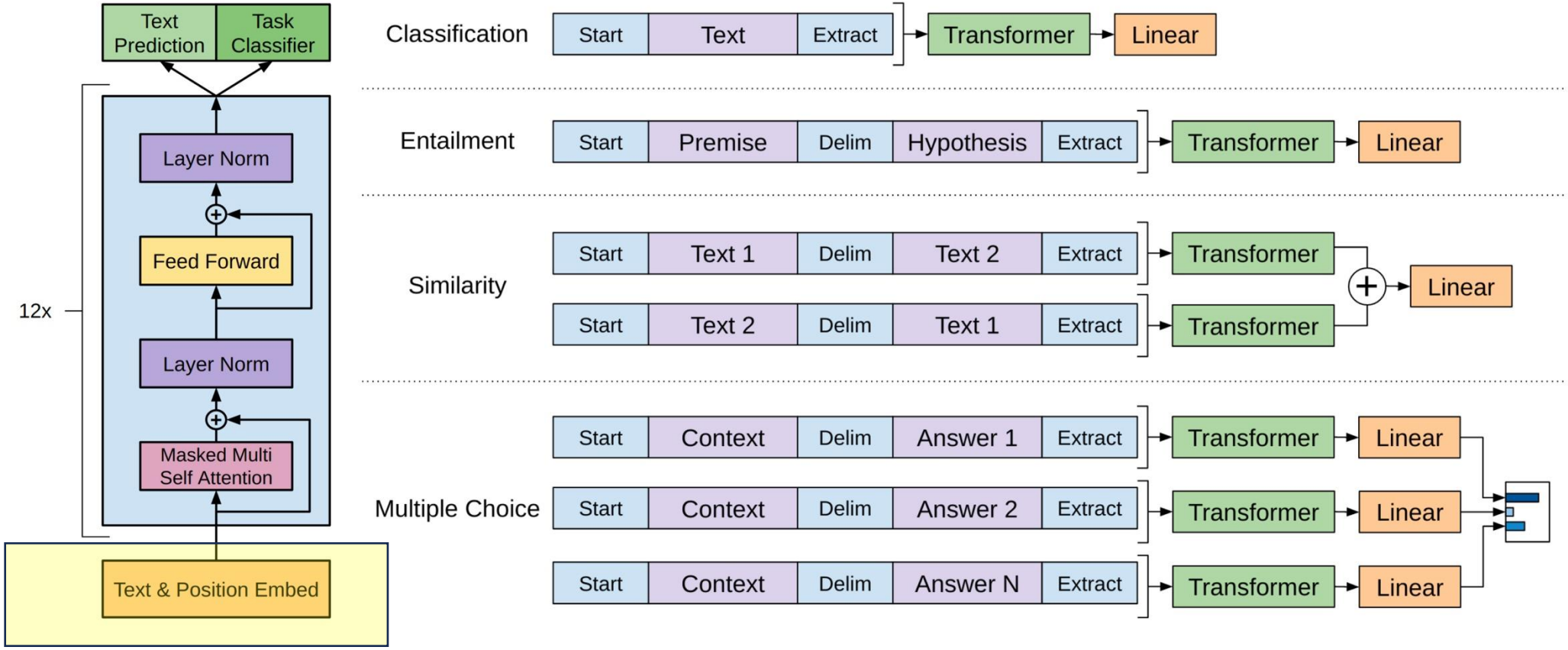
Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document …

Save Cite Cited by 17796 Related articles

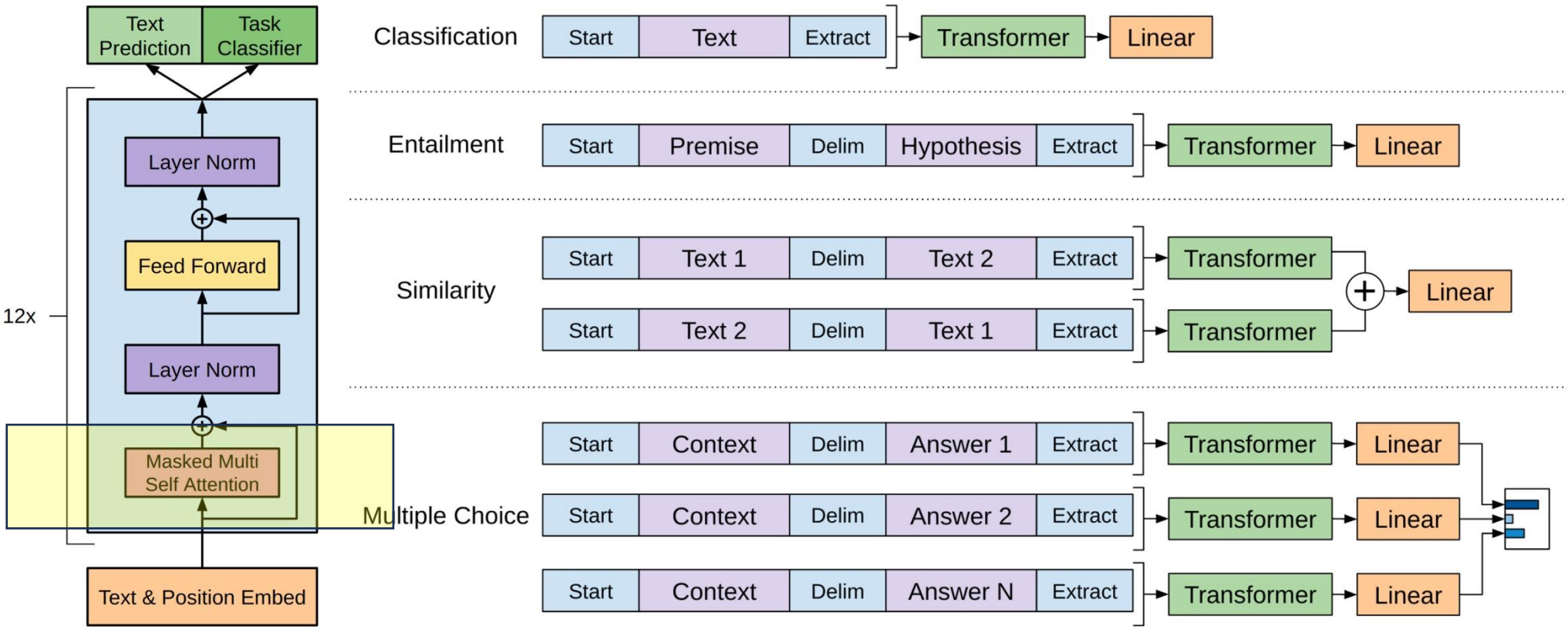
Base Model, Trained by Generative Pretraining



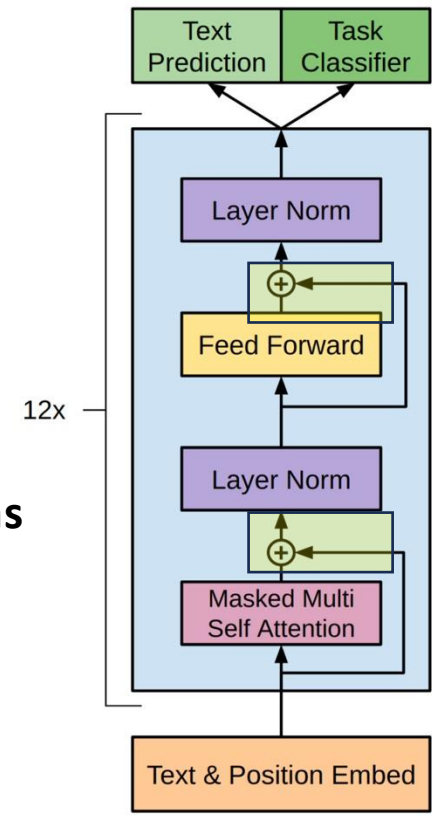
Word Embedding + Positional Embedding

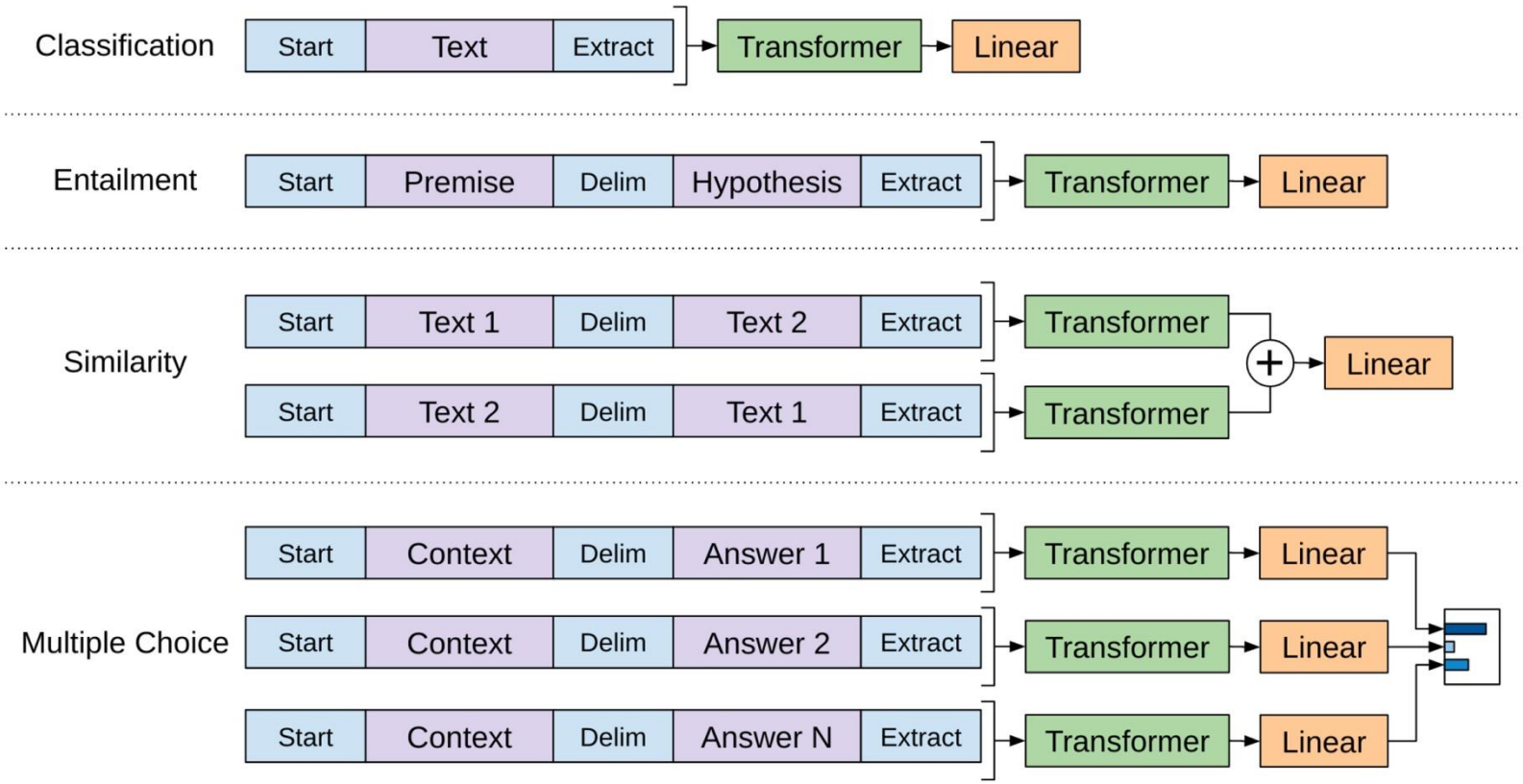


Multi-Head Attention

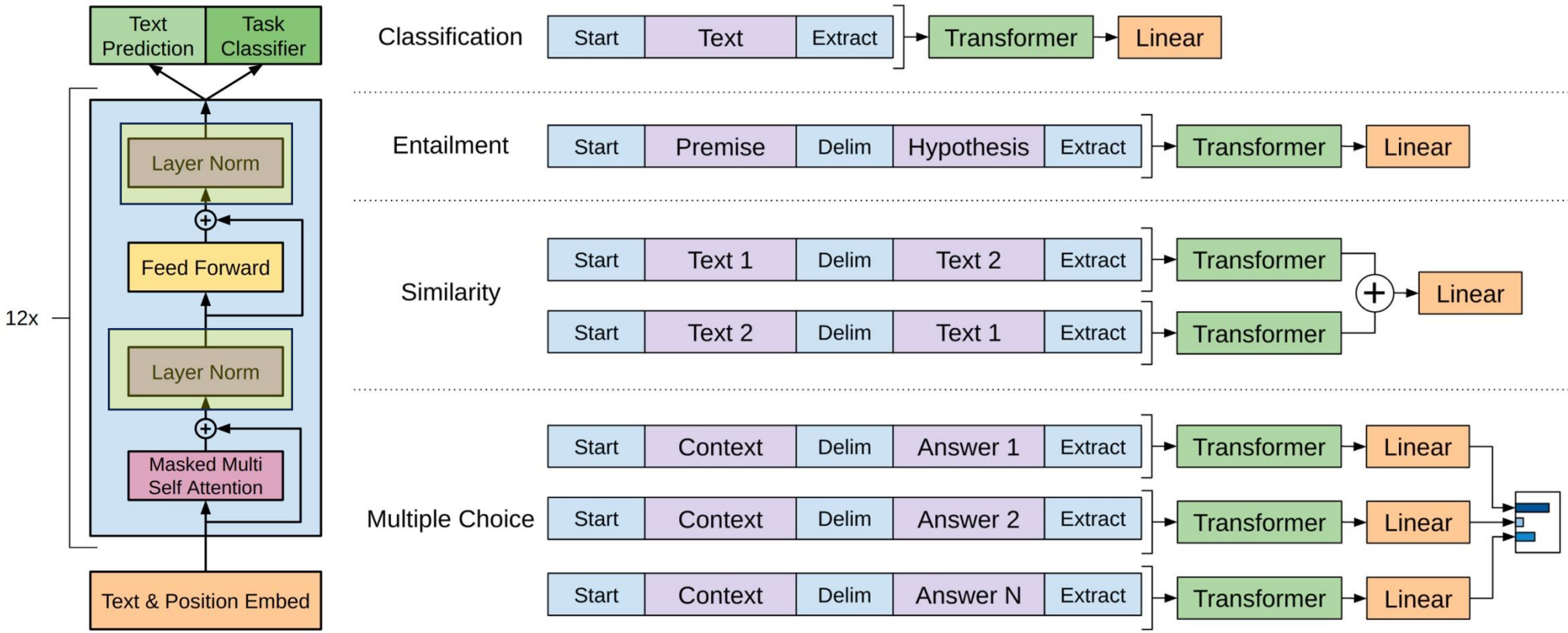


Skip Connections

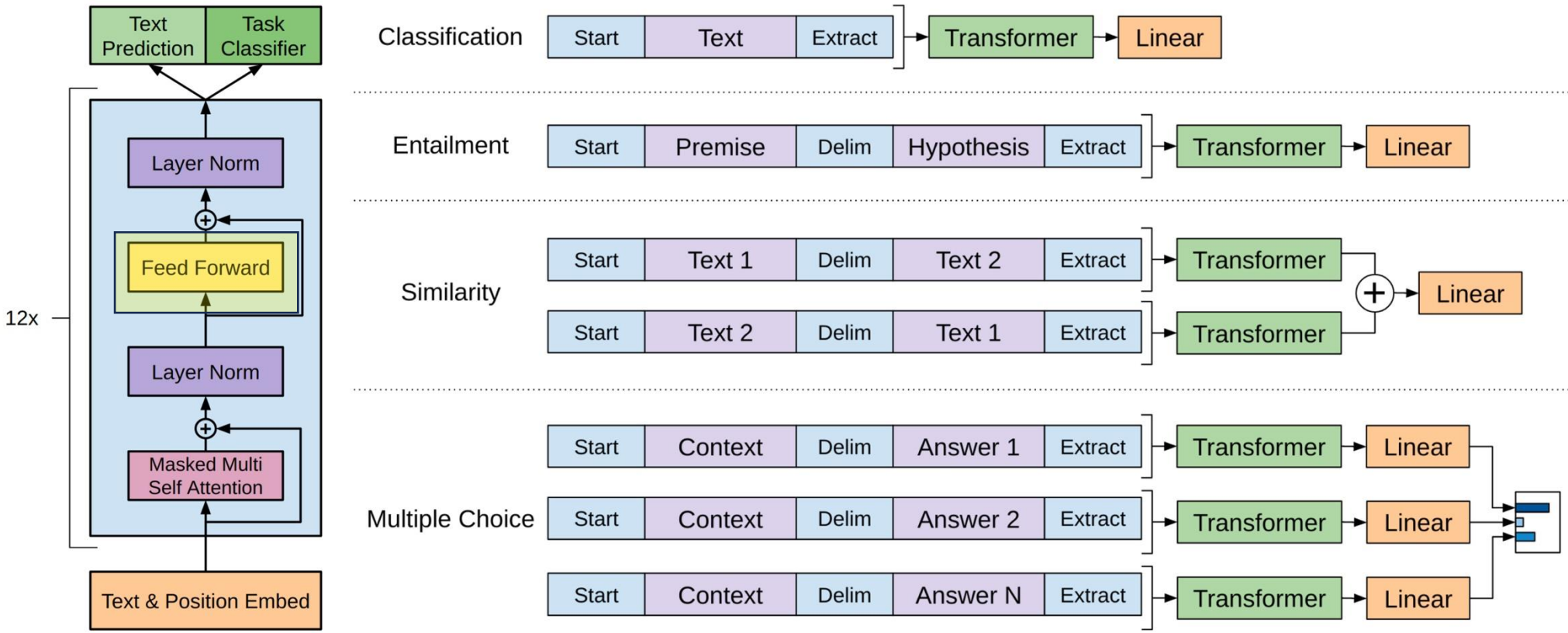




Normalization

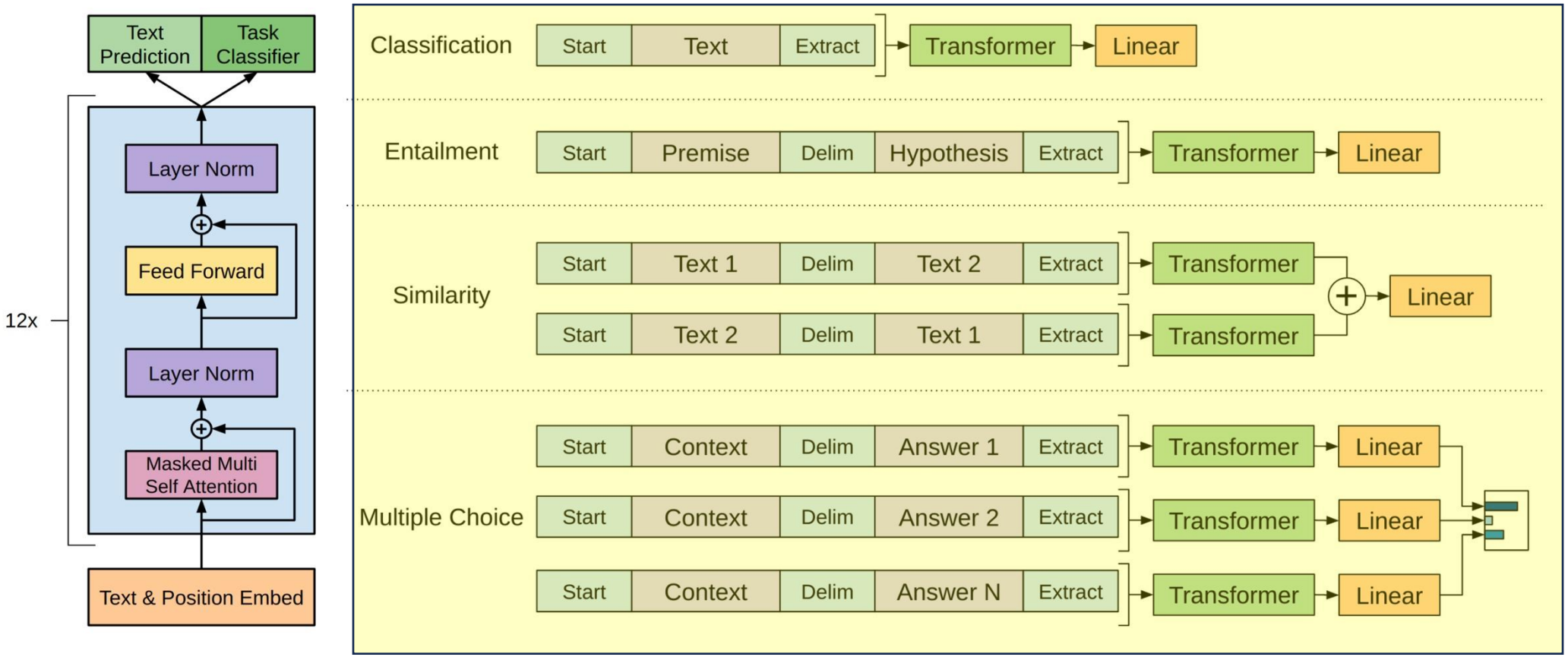


FF Network


Text Prediction
Task Classifier
Layer Norm
Feed Forward
Layer Norm
Masked Multi Self Attention
12x
Text & Position Embed
Classification
Start
Text
Extract
Transformer
Linear
Entailment
Start
Premise
Delim
Hypothesis
Extract
Transformer
Linear
Similarity
Start
Text 1
Delim
Text 2
Extract
Start
Text 2
Delim
Text 1
Extract
Transformer
Transformer
Linear
Multiple Choice
Start
Context
Delim
Answer 1
Extract
Start
Context
Delim
Answer 2
Extract
Start
Context
Delim
Answer N
Extract
Transformer
Linear

Down-Stream Tasks

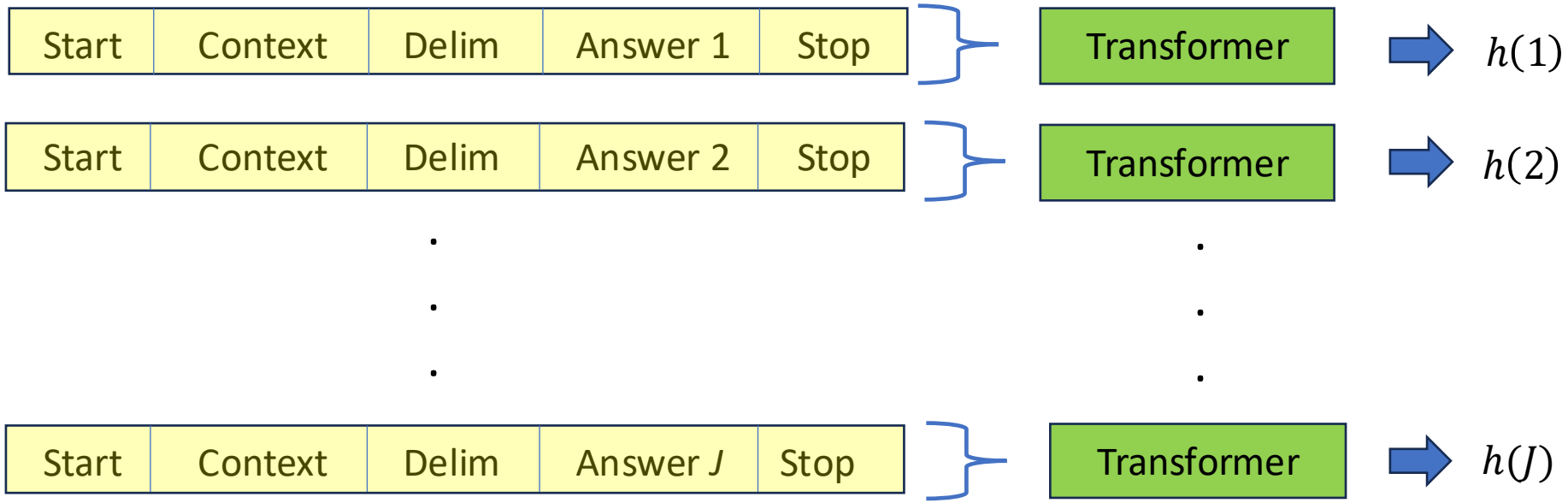
Refine (Fine-Tune) the Parameters of the Transformer
Append Task-Specific Linear Layer



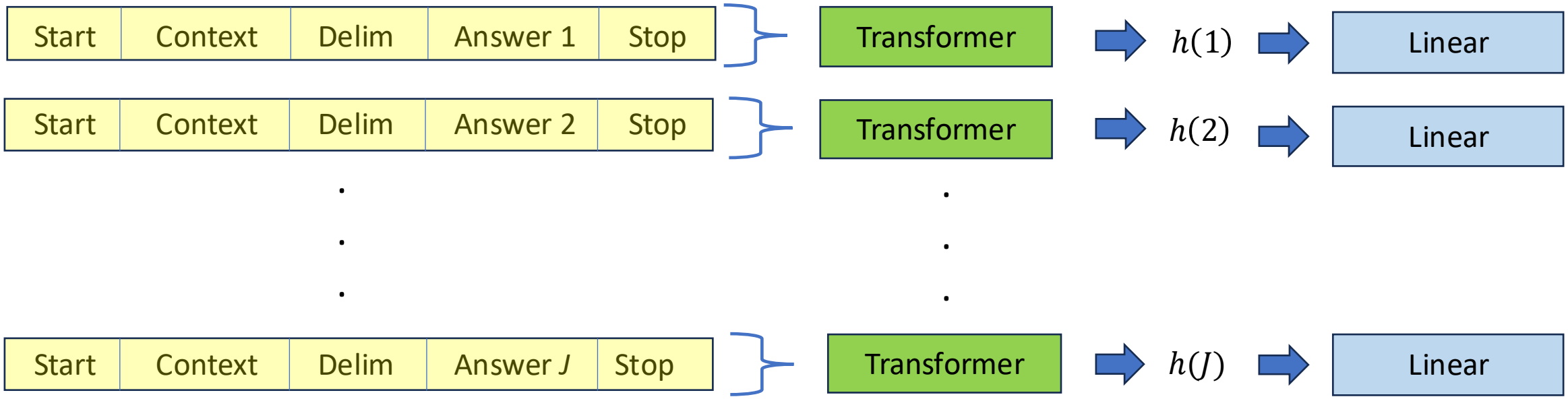
Task-Specific Linear Layer

- Let $h_N^{(L)}$ represent the vector output at the last layer (layer L), at the position corresponding to the last word in the prompt (could also possibly average vectors across all N positions)

- Consider the classification problem, with J possible categorical answers to a given question



Task-Specific Linear Layer

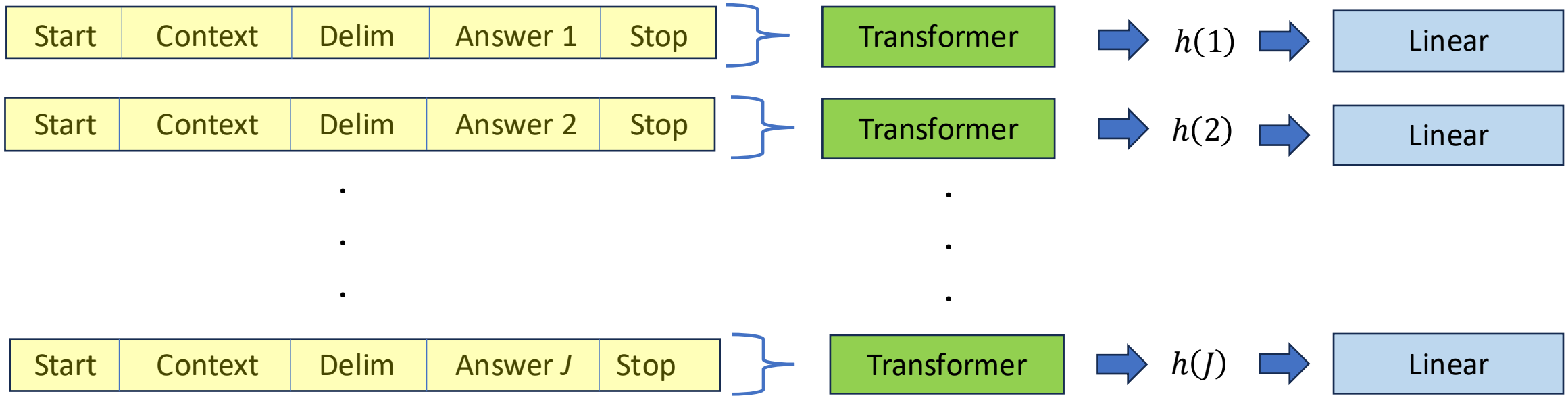


$$h(j) \in R^d \qquad j = 1, \dots, J$$

- Learn vector $w \in R^d$ with which we will implement a classifier

- Perform K inner products: $w^T h(1), \dots, w^T h(J)$

Task-Specific Linear Layer



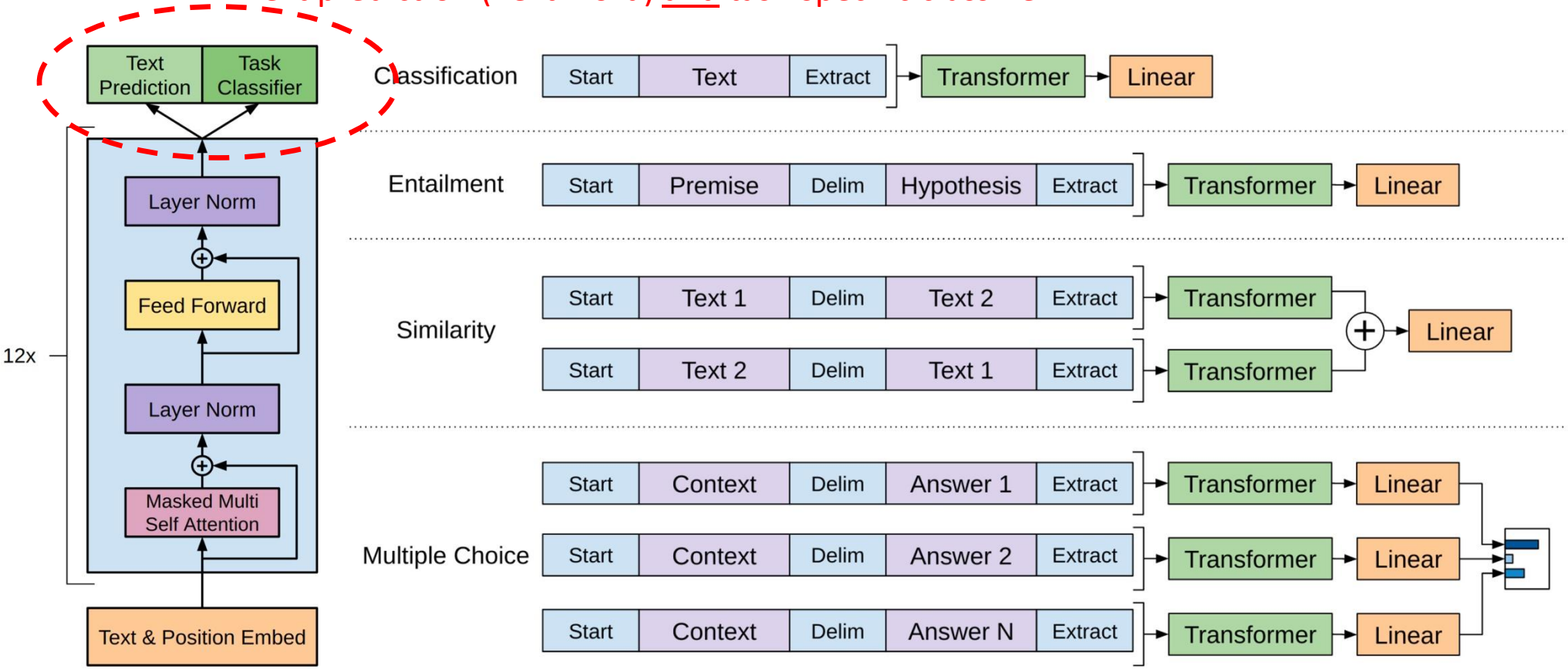
- Learn vector $w \in R^d$ with which we will implement a classifier
- Perform K inner products: $w^T h(1), \ldots, w^T h(J)$
- Train model such that the inner product associated with the true answer is largest

Task-Specific Linear Layer

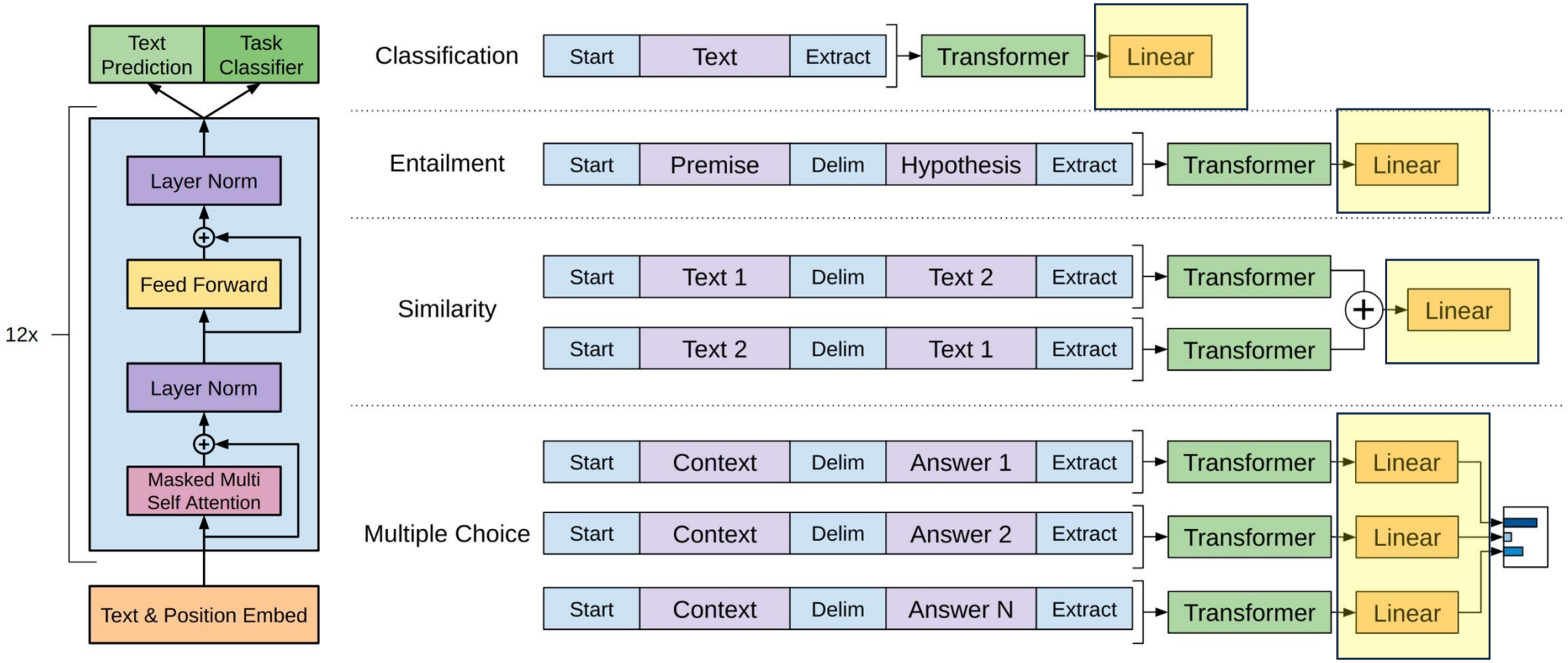
- Learn vector $w \in R^d$ with which we will implement a classifier
- Perform J inner products: $w^T h(1), \ldots, w^T h(J)$
- Train model such that the inner product associated with the true answer is largest
- Learned vector w defines a direction in feature space of alignment between the question and answer
- Learn w and refine the Transformer parameters, and apply the refined Transformer and linear w to new questions not seen previously

Down-Stream Tasks

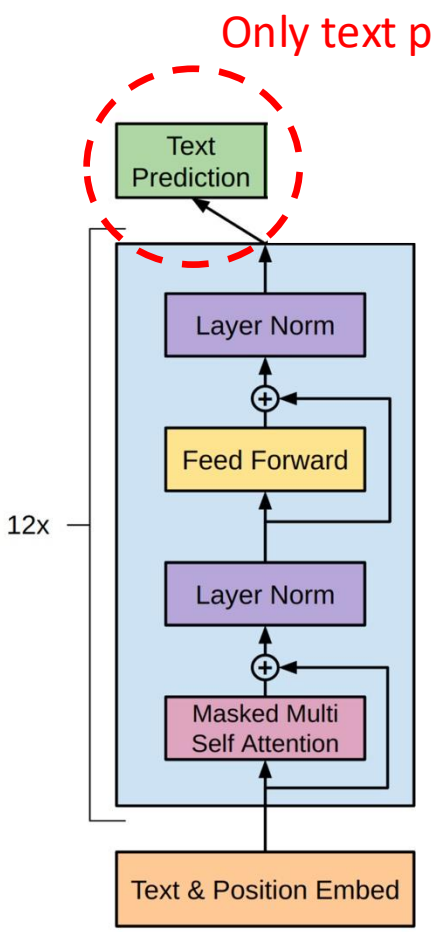
Text prediction (next word) and task-specific classifier



Linear Element at Output

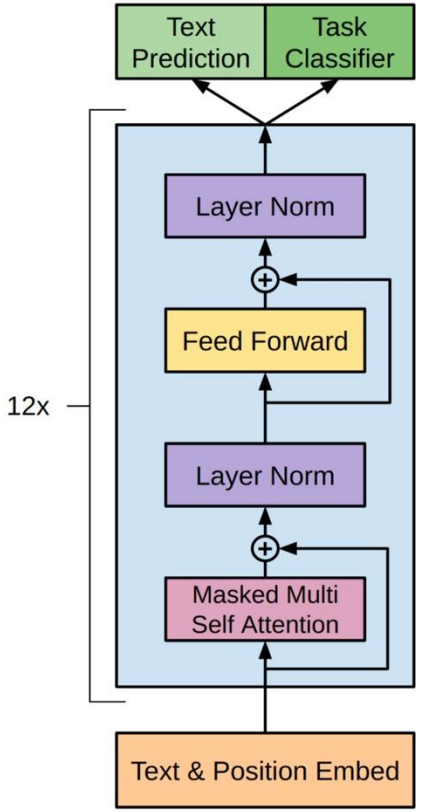


Pre-Training: Component for Predicting Text



- Assume given a sequence of tokens $U = \{u_1, \dots, u_n\}$, n large
- Let the Transformer parameters be represented as Θ
- We seek the language model $p(u_i | u_{i-1}, u_{i-2}, \dots, u_{i-k}; \Theta)$, where k is the context length
- Sliding window (mask) defines series of context blocks
- Predict each token u_i based on preceding k tokens $(u_{i-1}, \dots, u_{i-k})$

Model Construction

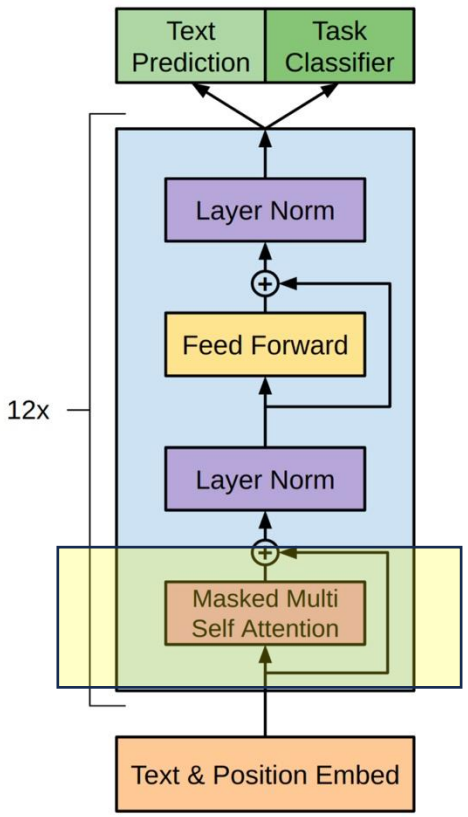


- Design with a single "stack" is called a multi-layer Transformer decoder
- Employs multi-head attention
- If the sequence of vectors input to an attention layer is $w_1, \dots w_k$ (context of length k), then the jth head, $j = 1, \dots, J$, employs projections

$$W_j^Q w_i \qquad W_j^K w_i \qquad W_j^V w_i$$

for $i = 1, \dots, k$

Model Construction



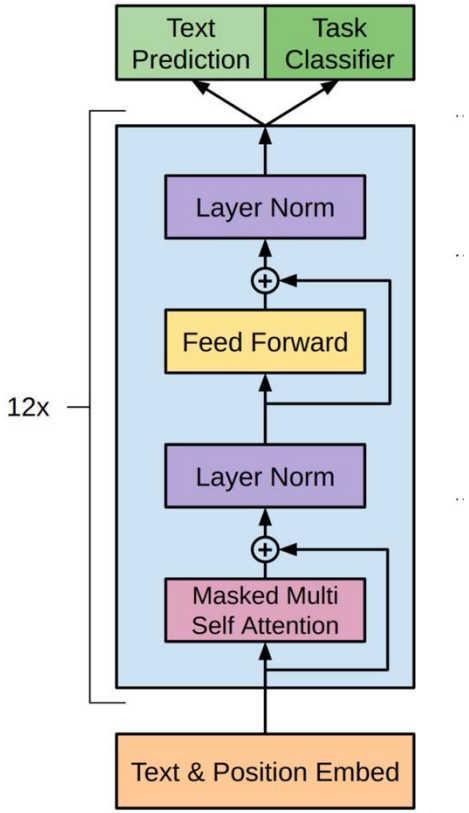
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_{n'}\right)}$$

- For simplicity, in above we sum over all keys

- <u>Masked</u> self attention: Attend only to k keys of tokens coming *prior* to given tokens/keys in the sequence

Model Construction



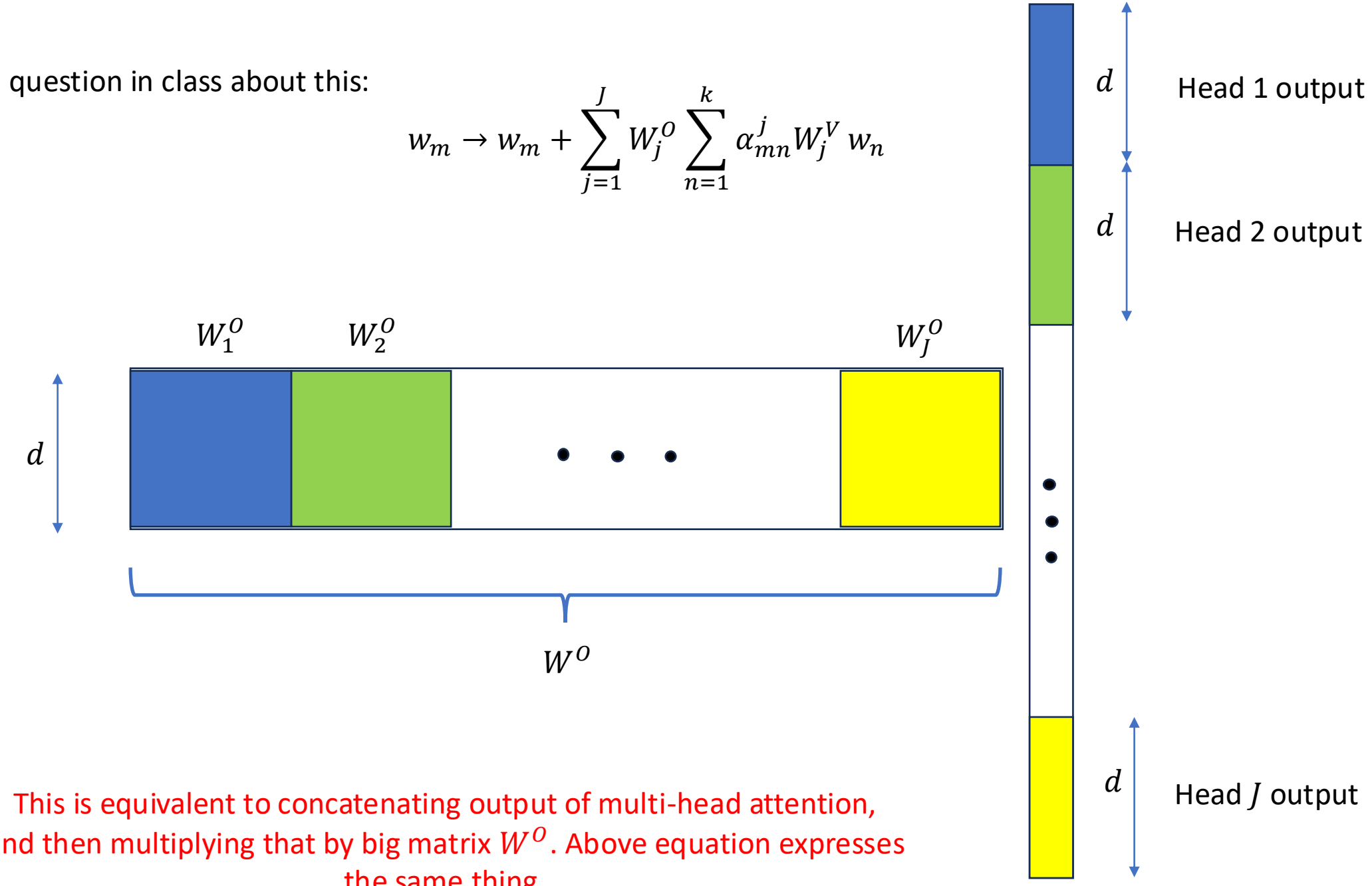
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \underbrace{\left(W_j^Q\right)^T W_j^K}_{C_j} w_{n'}\right)}$$

$$w_m \rightarrow w_m + \sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n$$

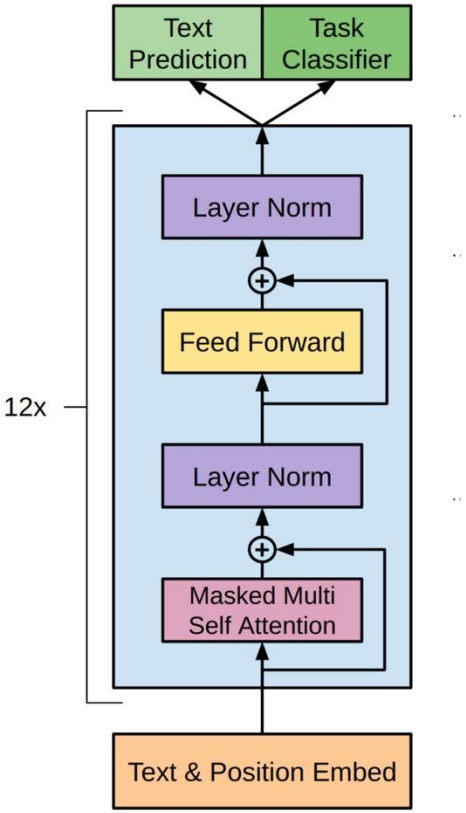
There was a question in class about this:

$$w_m \rightarrow w_m + \sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^j W_j^V w_n$$



This is equivalent to concatenating output of multi-head attention, and then multiplying that by big matrix W^O. Above equation expresses the same thing.

Model Construction



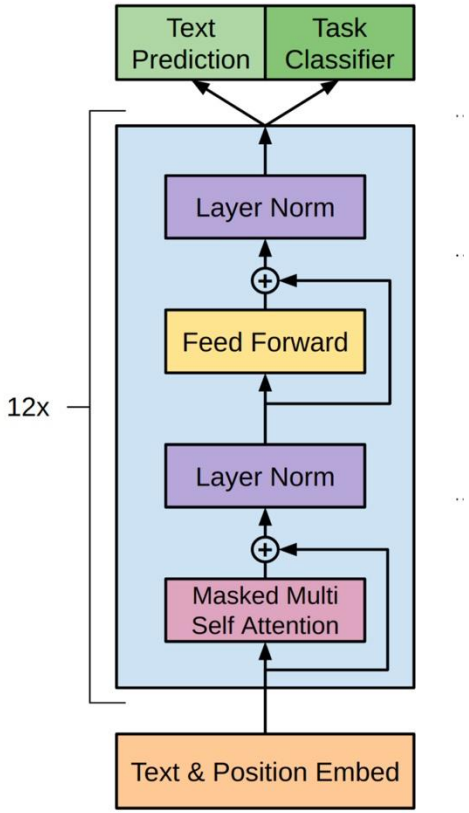
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \underbrace{\left(W_j^Q\right)^T W_j^K}_{C_j} w_{n'}\right)}$$

$$w_m \to w_m + \sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n$$

Attention paid to key m for key n, head j

Model Construction



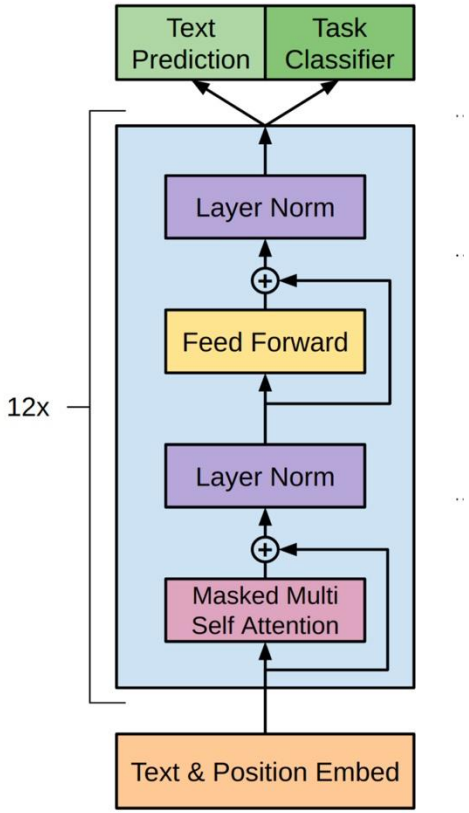
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \underbrace{\left(W_j^Q\right)^T W_j^K}_{C_j} w_{n'}\right)}$$

$$w_m \rightarrow w_m + \sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n$$

Value associated with key n, head j

Model Construction



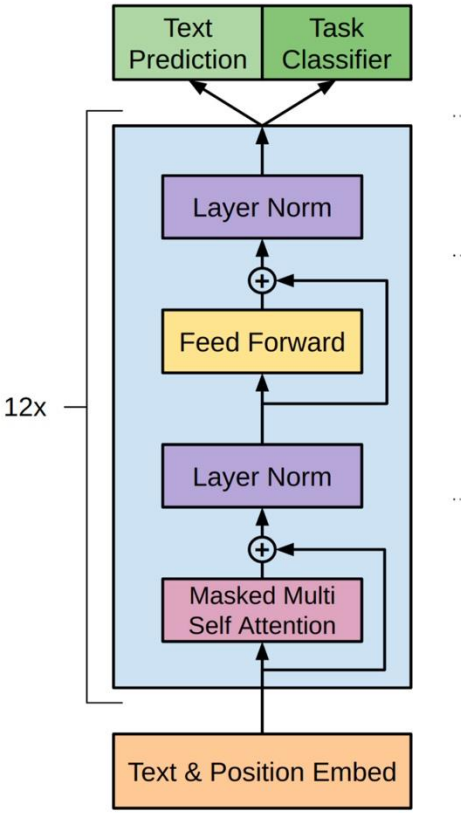
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \underbrace{\left(W_j^Q\right)^T W_j^K}_{C_j} w_{n'}\right)}$$

$$w_m \rightarrow w_m + \sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n$$

Aggregate J attention heads

Model Construction



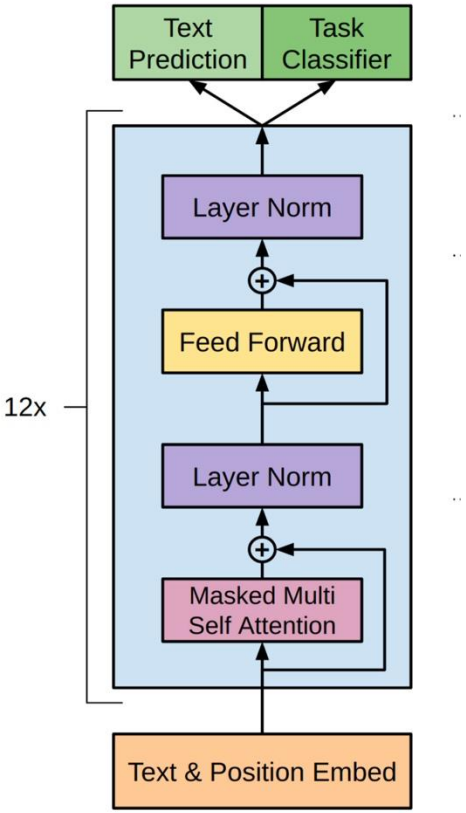
- Softmax attention weights for query m and key n:

$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_{n'}\right)}$$

Softmax Attention

$$w_m \rightarrow w_m + \boxed{\sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n}$$

Model Construction



- Softmax attention weights for query m and key n:

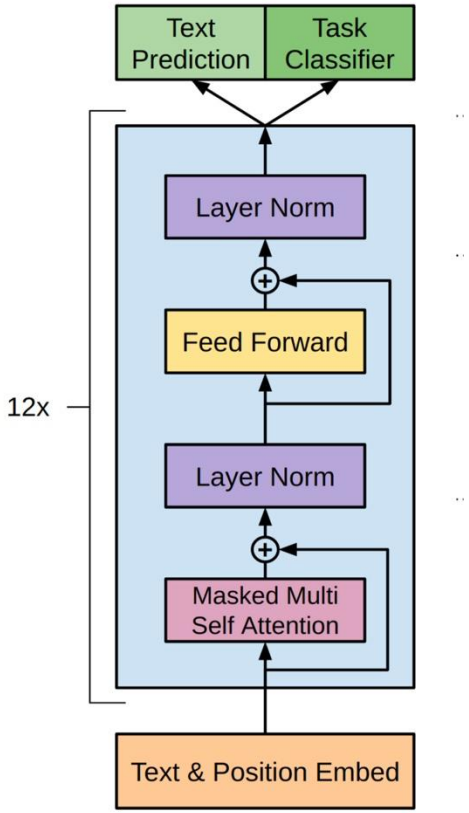
$$\alpha_{mn}^{j} = \frac{\exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_n\right)}{\sum_{n'=1}^{k} \exp\left(w_m^T \left(W_j^Q\right)^T W_j^K w_{n'}\right)}$$

Softmax Attention

Skip Connection

$$w_m \rightarrow \boxed{w_m} + \boxed{\sum_{j=1}^{J} W_j^O \sum_{n=1}^{k} \alpha_{mn}^{j} W_j^V w_n}$$

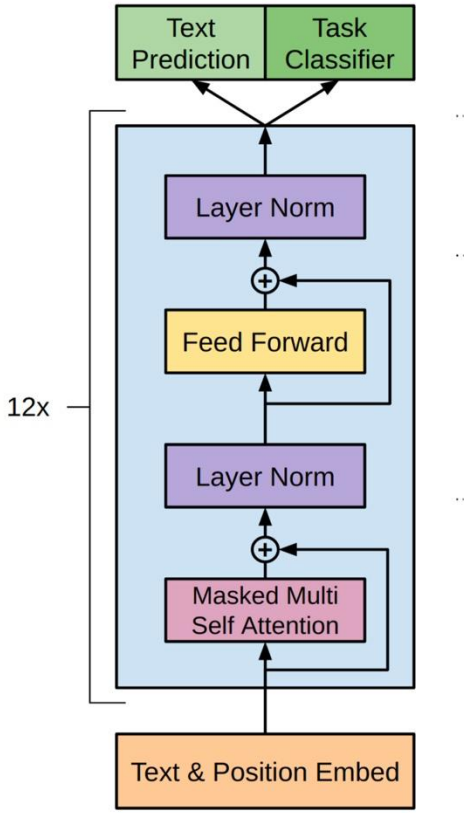
Model Construction



- If the *sequence* of vectors output from Transformer block $l-1$ is denoted h_{l-1} then the output is represented as

$$h_l = \text{transformer_block}(h_{l-1})$$

Model Construction



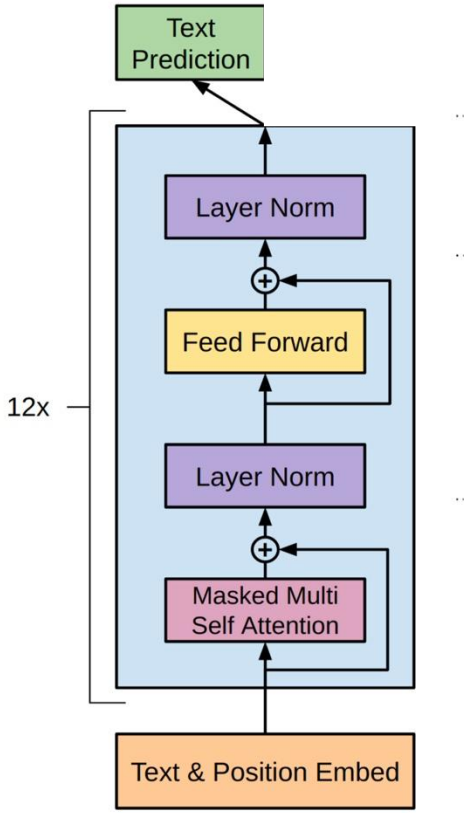
- If the *sequence* of vectors output from Transformer block $l-1$ is denoted h_{l-1} then the output is represented as

$$h_l = \text{transformer_block}(h_{l-1})$$

- Next word predicted using final vector output above last word in the sequence before new word

- Softmax based on that last output vector

Pre-Training



- The embedding at the lowest layer (input) is

$$h_0 = W_e U + W_p$$

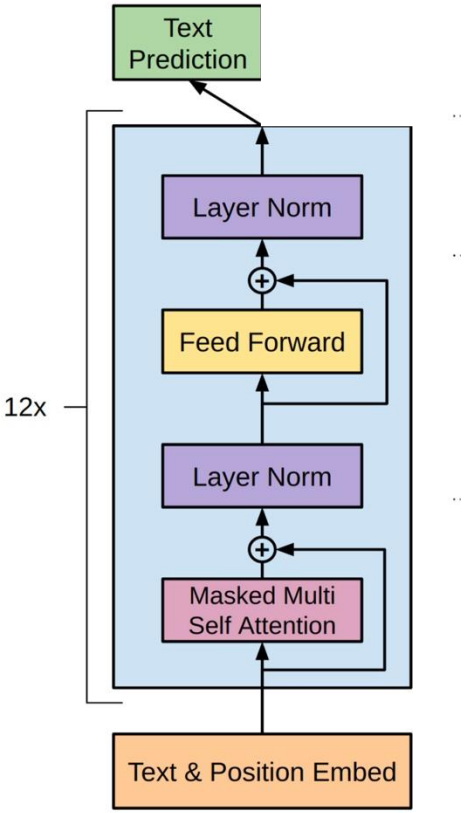
- Matrix U has columns corresponding to each of the words in the sequence and the columns are ``one hot'' vectors

- Dimension of columns of U equals number of tokens/words

- Each column is all-zeros, except for a single element denoting the word location in the codebook; that element has value 1

- W_e is the codebook, with each column representing a word-meaning embedding vector; number of columns = number of tokens in codebook

Pre-Training



- The embedding at the lowest layer (input) is

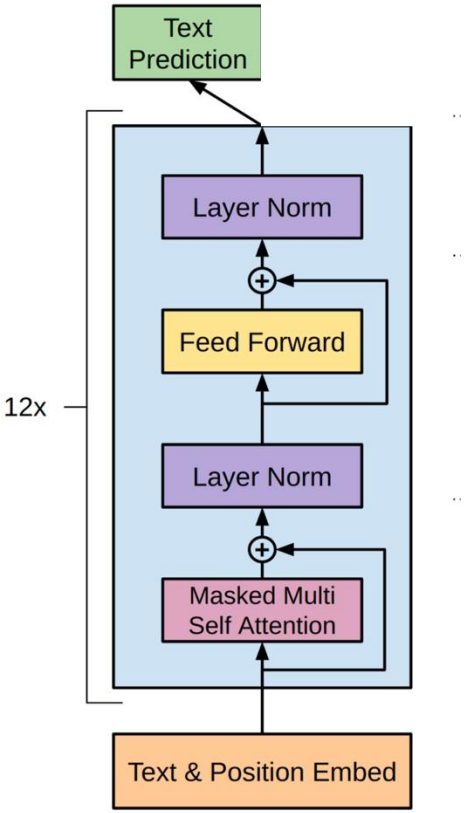
$$h_0 = W_e U + W_p$$

- Matrix W_p represents positional embeddings, with columns corresponding to positions

- We input all words in, but with masked attention, only attend to the k preceding words

- Final output at each position corresponds to Transformer analysis of a given k-dimensional context, and we wish for the model to predict next word after the context (masked attention)

Pre-Training



- The embedding at the lowest layer (input) is

$$h_0 = W_e U + W_p$$

- For n Transformer layers, implement

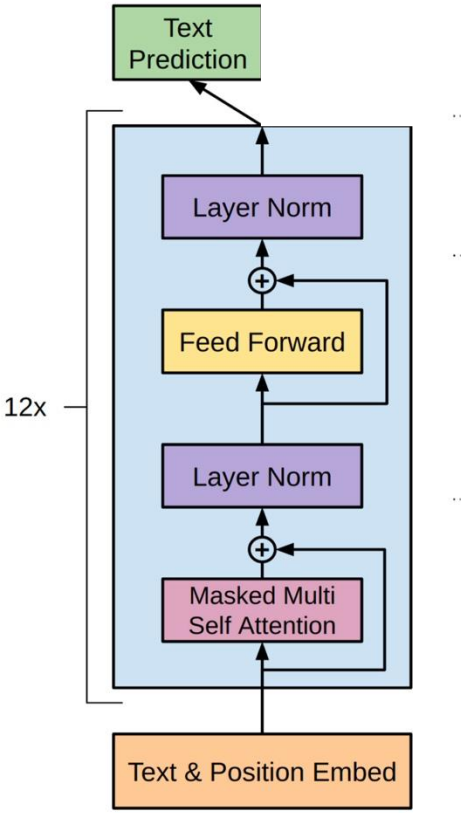
$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

- The output layer is h_L

- Let $h_{L,i}$ represent the output vector for word i in the sequence

- Probability of next word is $p(u_{i+1} | u_i, \dots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$

Pre-Training



- The embedding at the lowest layer (input) is

$$h_0 = W_e U + W_p$$

- For n Transformer layers, implement

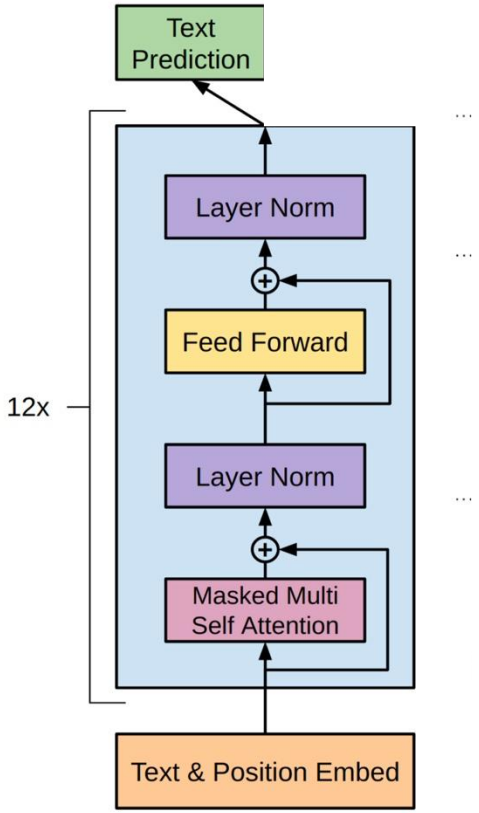
$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

- The output layer is h_L

- Let $h_{L,i}$ represent the output vector for word i in the sequence

- Probability of next word is $p(u_{i+1} | u_i, \ldots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$

Pre-Training

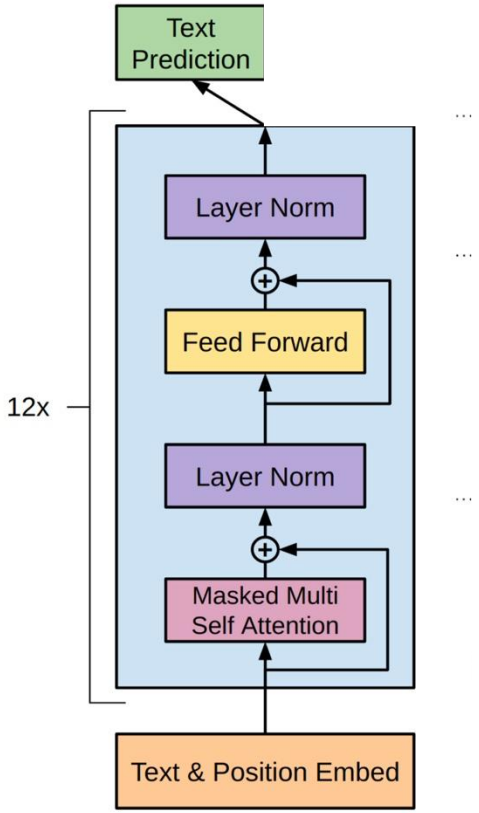


$$h_0 = W_e U + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

$$p(u_{i+1} | u_i, \dots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$$

Pre-Training



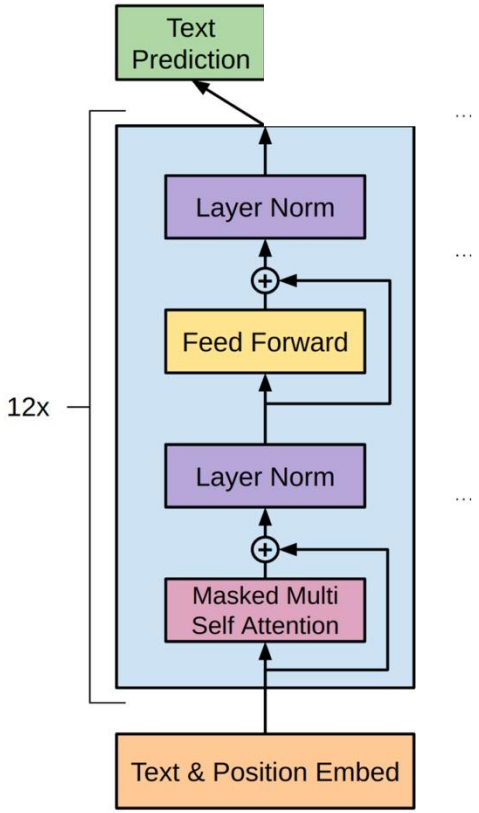
$$h_0 = W_e U + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

$$p(u_{i+1} | u_i, \dots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$$

- ❑ Pretrain for all text in corpus
- ❑ Masked self attention, with constant context length
- ❑ Output is the contextual coding for all contiguous k-dimensional words
- ❑ Use output preceding it to predict probability of next word

Pre-Training



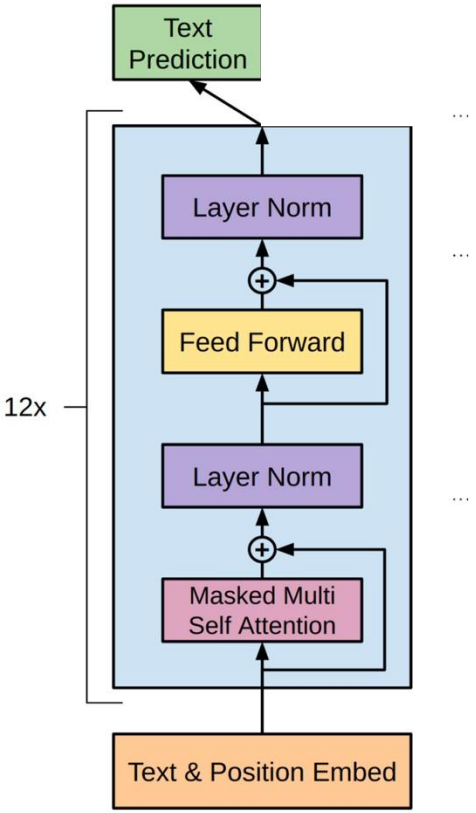
$$h_0 = W_e U + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

$$p(u_{i+1} | u_i, \ldots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$$

- ❑ Builds a very good initialization of model parameters
- ❑ These parameters serve as initialization for subsequent tasks
- ❑ For such (labeled/supervised) tasks, we may not have as much data
- ❑ Pretraining critical for good initialization

Model Parameters Learned in Pre-Training



$$h_0 = W_e U + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}), \quad \forall\ l \in [1, L]$$

$$p(u_{i+1} | u_i, \dots, u_{i-k+1}) = \text{softmax}(W_e^T h_{L,i})$$

- Word embedding vectors: W_e

- Projection matrices for each head, at each attention layer, and the aggregation matrix

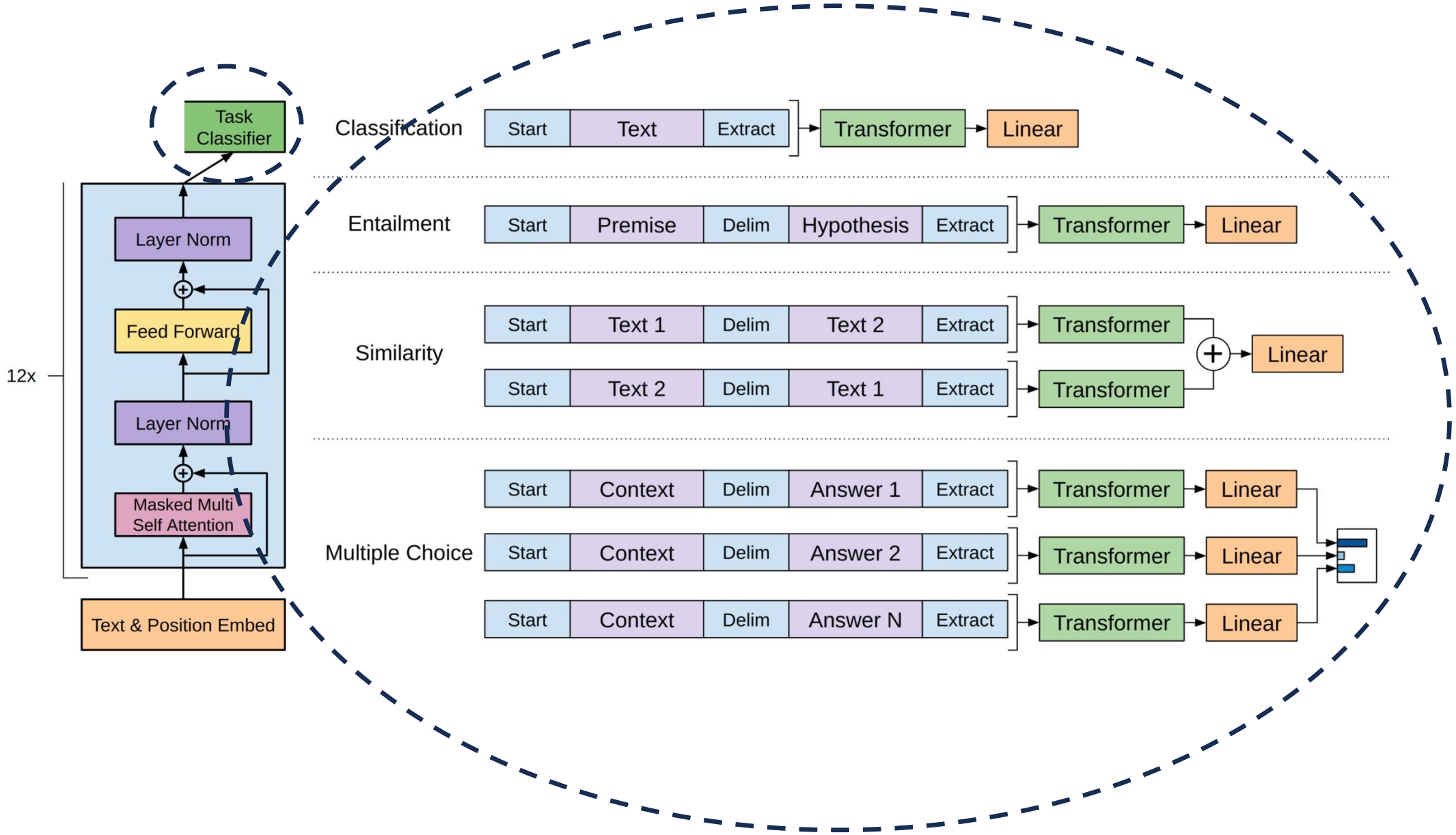
$$W_{lj}^Q \qquad W_{lj}^K \qquad W_{lj}^V \qquad W_l^O$$

$$l = 1, \dots, L \qquad j = 1, \dots, n_l^H$$

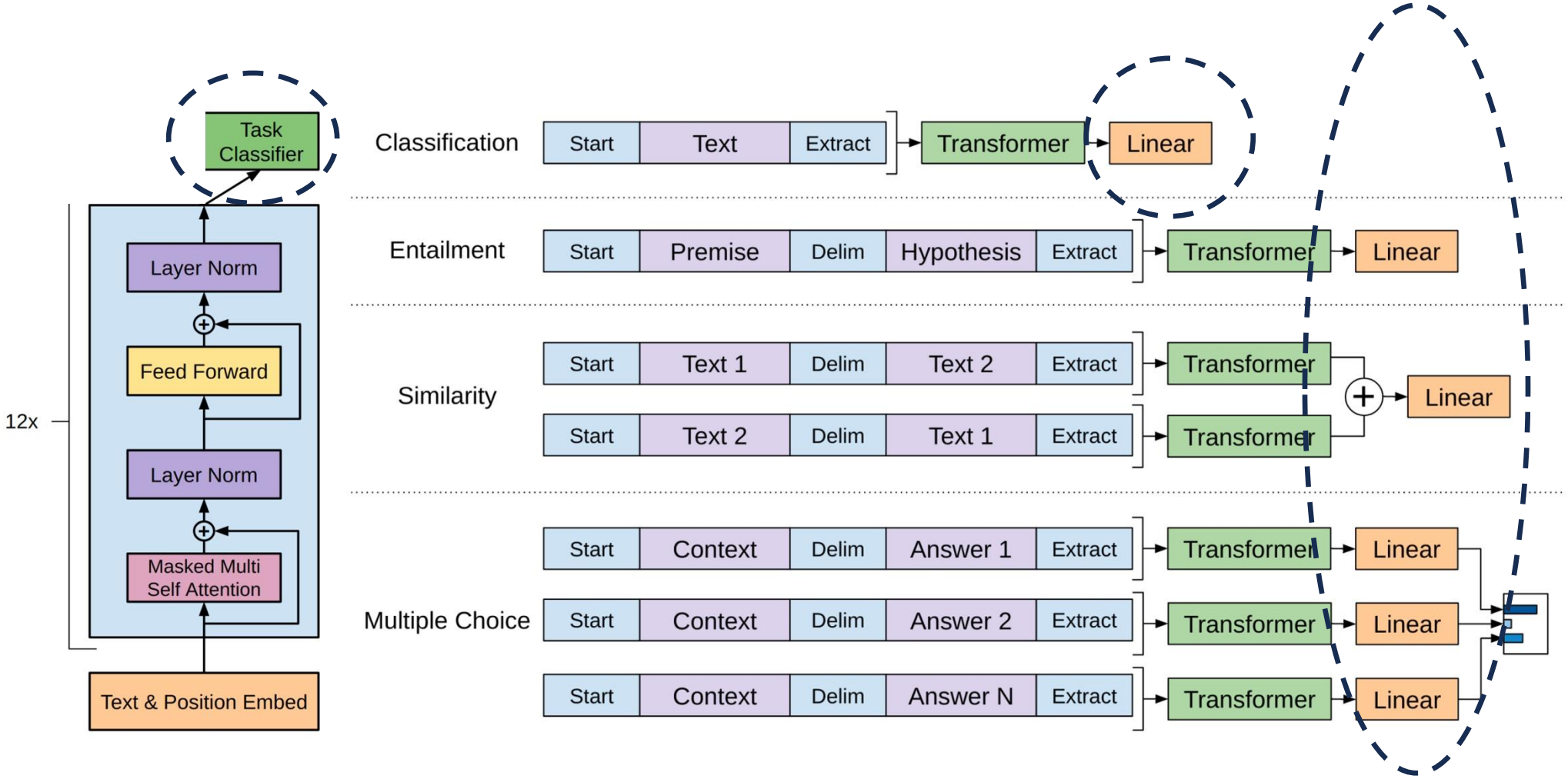
- Feedforward matrices and biases

$$F_l \qquad b_l \qquad l = 1, \dots, L$$

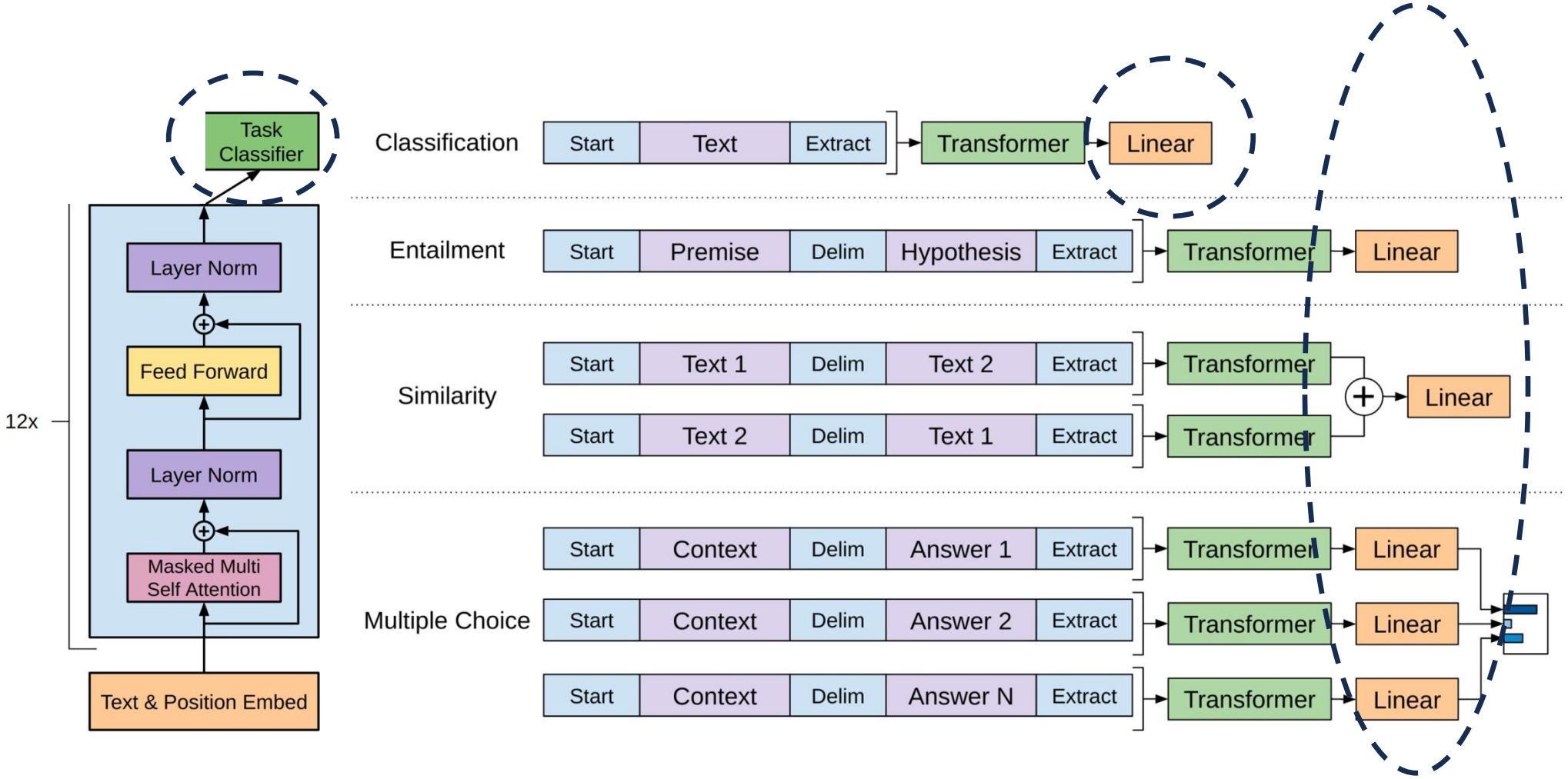
Down-Stream Task(s)



Down-Stream Task(s)

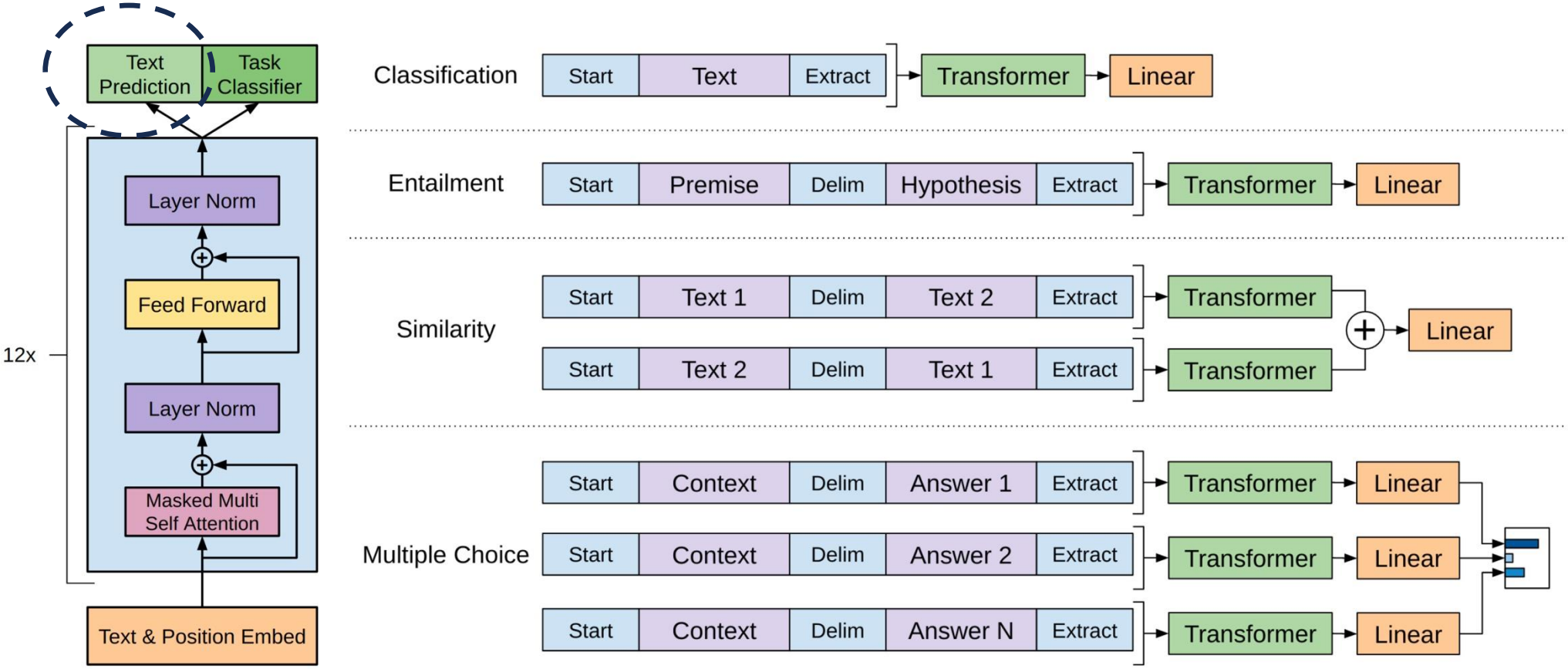


Down-Stream Task(s)



Linear Layer Appended to the Output of Transformer
Instead of Predict Next Word

Down-Stream Task(s)



Found Useful to Also Perform Masked-Attention Prediction of Next Word on New Task

Down-Stream Task(s): Supervised Fine-Tuning

- Assume that we are given labeled data $D_s = \{(x_1^i, \dots, x_m^i, y^i)\}_{i=1,N}$

- Let h_L^m be the Transformer output at layer L (last attention layer), for the mth position in the sequence

- The linear layer represented by vector w

- Probability of categorical output $y \in \{1, \dots, J\}$ is

$$p(y_i | x_1^i, \dots., x_m^i) = \mathrm{softmax}(w^T h_{L,1}, \dots, w^T h_{L,J})$$

Supervised Fine-Tuning

- Assume that we are given labeled data $D_s = \{(x_1^i, \dots, x_m^i, y^i)\}_{i=1,N}$

$$p(y_i | x_1^i, \dots., x_m^i) = \text{softmax}(w^T h_{L,1}, \dots, w^T h_{L,J})$$

- Loss function for labeled data:

$$\mathcal{L}_2(D_s) = -\sum_{D_s} \log p(y^i | x_1^i, \dots, x_m^i; \boxed{\Theta, W_y})$$

- Predict-next-word loss

$$\mathcal{L}_1(D_s) = -\sum_i \log p(u_i | u_{i-k}, \dots, u_{i-1}; \boxed{\Theta})$$

Supervised Fine-Tuning

- Assume that we are given labeled data $D_s = \{(x_1^i, \dots, x_m^i, y^i)\}_{i=1,N}$

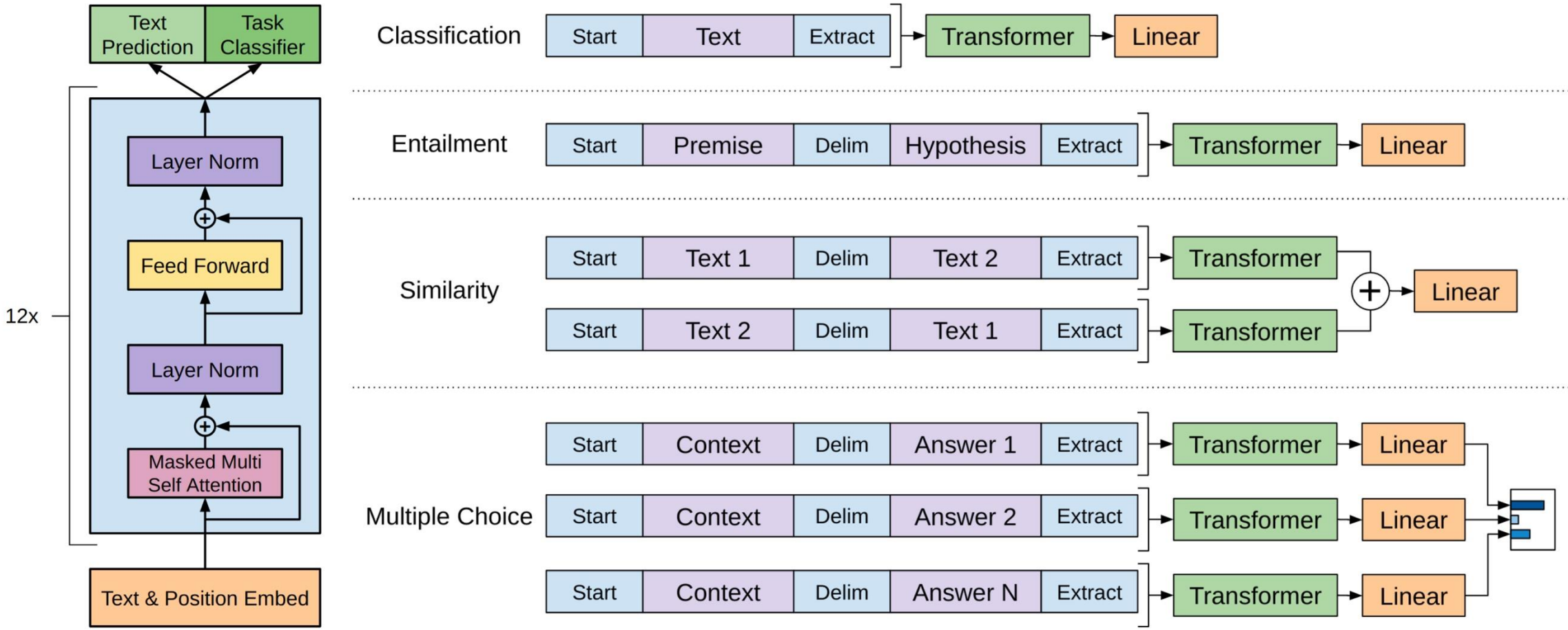
$$p(y_i | x_1^i, \dots., x_m^i) = \text{softmax}(W_y h_L^m)$$

$$\mathcal{L}_1(D_s) = -\sum_i \log p(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

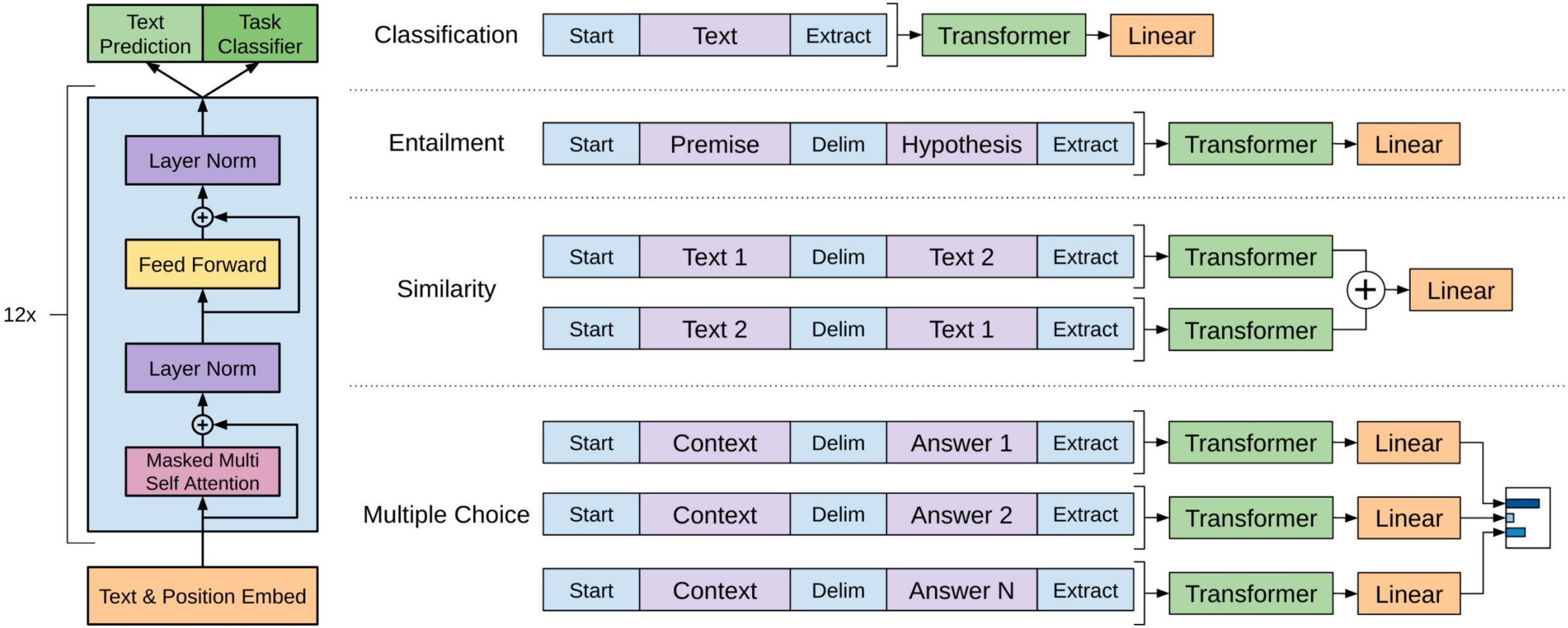
$$\mathcal{L}_2(D_s) = -\sum_{D_s} \log p(y^i | x_1^i, \dots, x_m^i; \Theta, W_y)$$

- Total loss function:

$$\mathcal{L}_3(D_s) = \mathcal{L}_2(D_s) + \lambda \mathcal{L}_1(D_s)$$


Text Prediction
Task Classifier
Layer Norm
Feed Forward
Layer Norm
Masked Multi Self Attention
12x
Text & Position Embed
Classification
Start
Text
Extract
Transformer
Linear
Entailment
Start
Premise
Delim
Hypothesis
Extract
Transformer
Linear
Similarity
Start
Text 1
Delim
Text 2
Extract
Start
Text 2
Delim
Text 1
Extract
Transformer
Transformer
Linear
Multiple Choice
Start
Context
Delim
Answer 1
Extract
Start
Context
Delim
Answer 2
Extract
Start
Context
Delim
Answer N
Extract
Transformer
Linear

Subsequent Versions of GPT



In many ways like GPT-1, except that larger models are used

Bidirectional Encoder Representations from Transformers (BERT)

- The heart of GPT-style pre-training is based on predicting the next word
- This leads to an autoregressive framework, with masked attention
- Do not attend to words after the word to be predicted
- Autoregressive training natural, but a consequence of the way pretraining is done
- If we do pre-training on tasks other than predicting the next word, we may generalize how attention is implemented

Bidirectional Encoder Representations from Transformers (BERT)

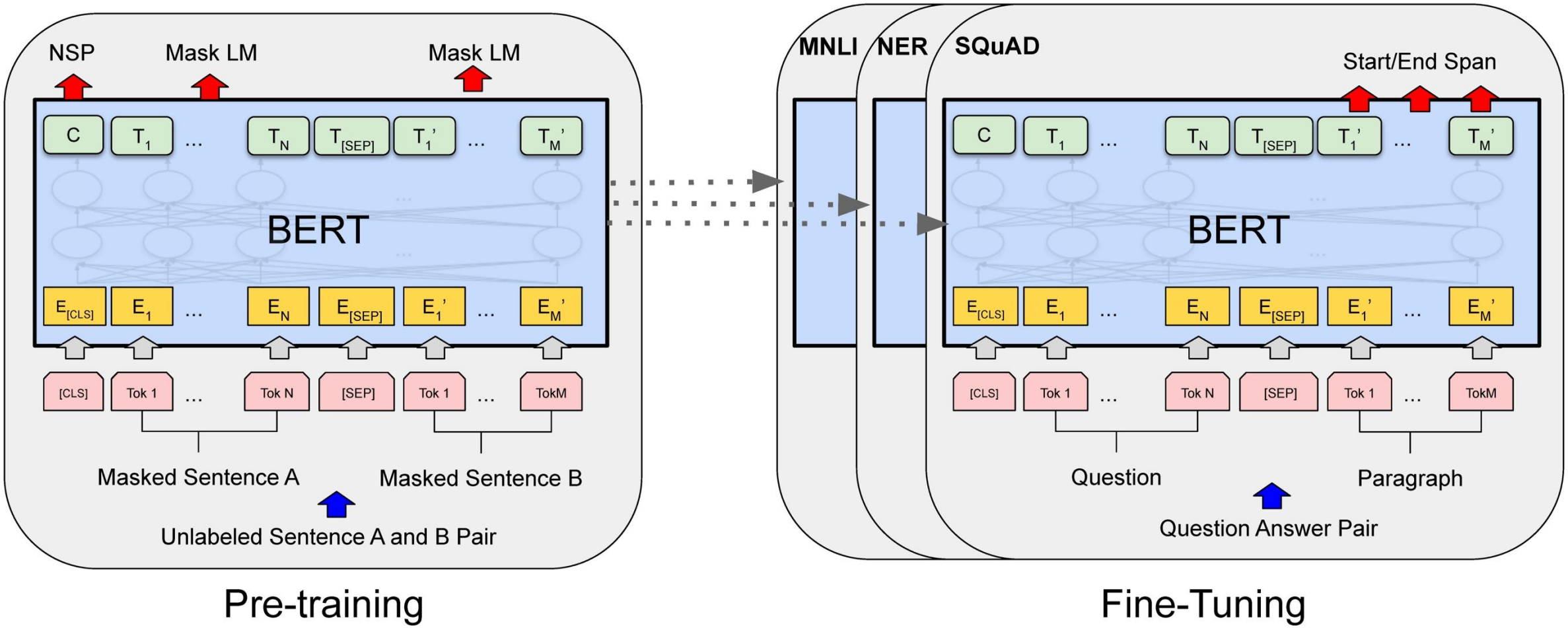
- Tasks that may be considered:

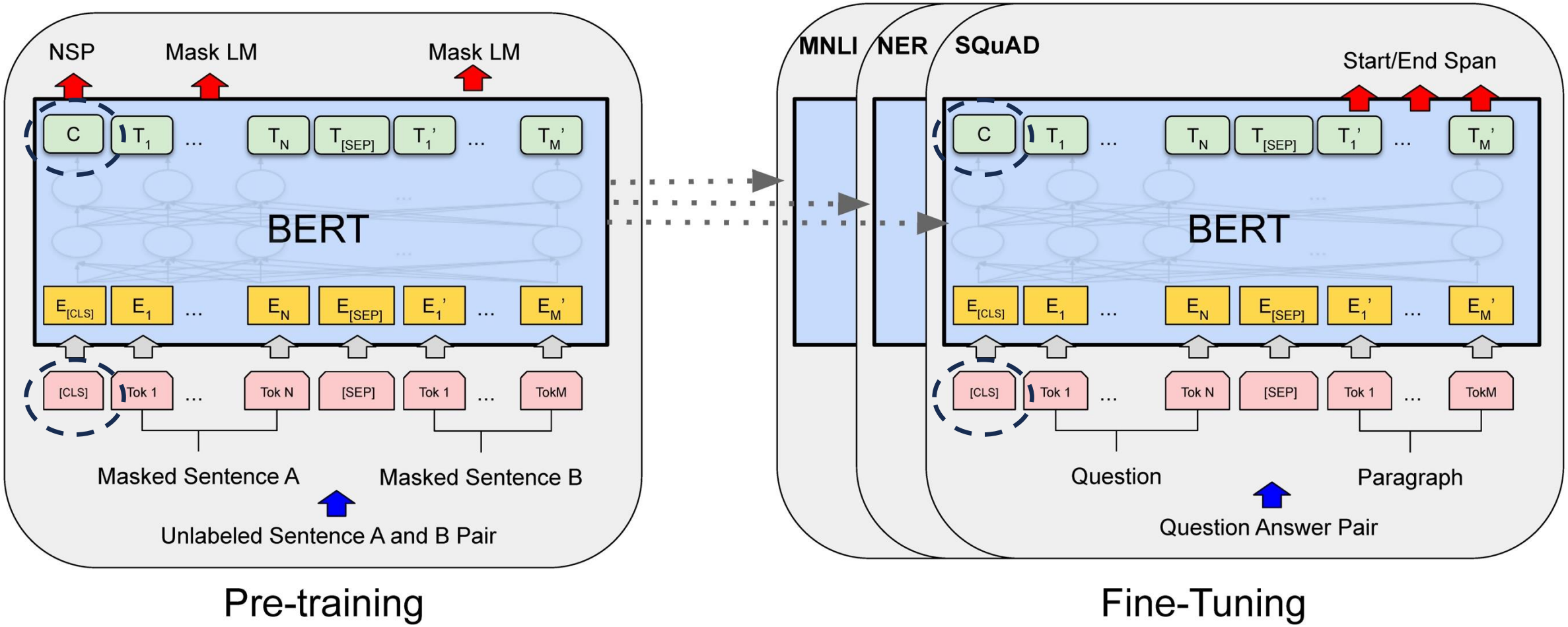
  - Model learns to predict whether two consecutive given sentences actually followed each other in text (NSP: next-sentence prediction)

  - Simple supervised task that may be constituted from unlabeled documents

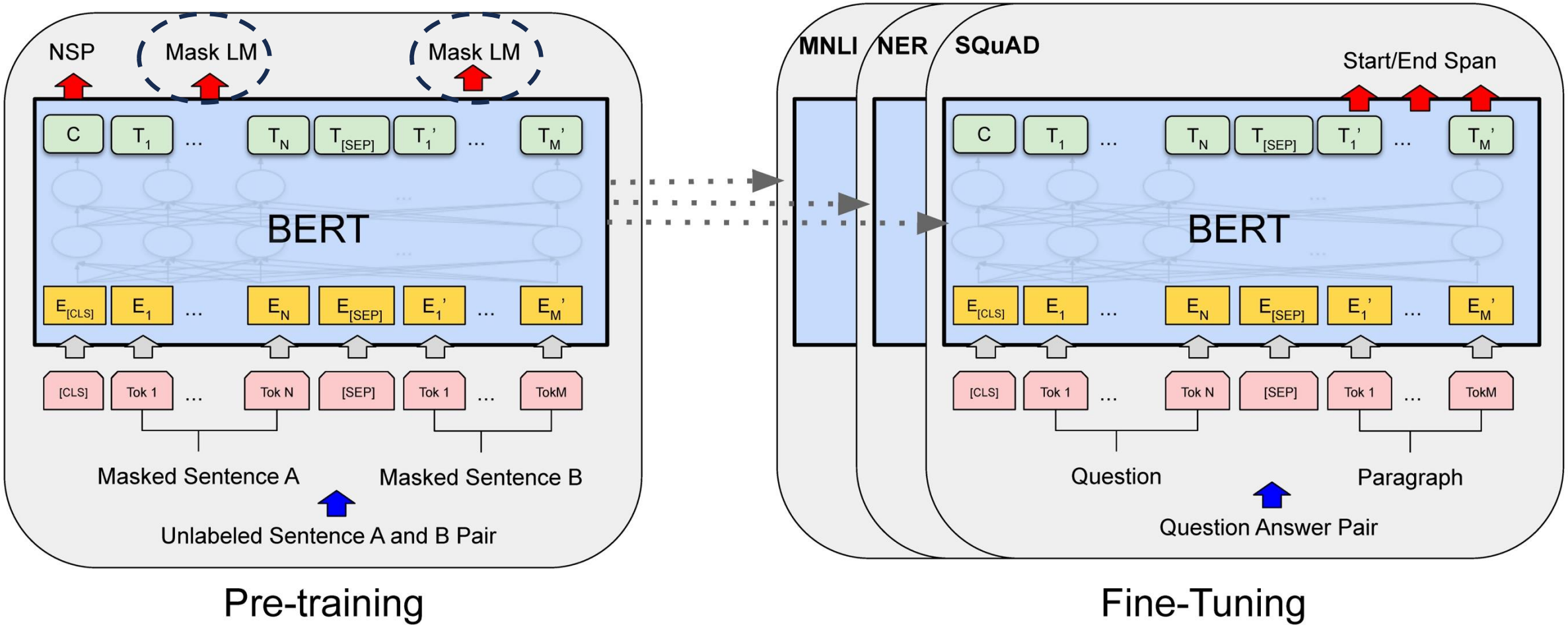
  - Similarly, like in CBOW (but now accounting for word order), we can mask out a fraction of the words at random

  - Transformer tasked with predicting missing word (*not* the next word); masked Language Model (LM)

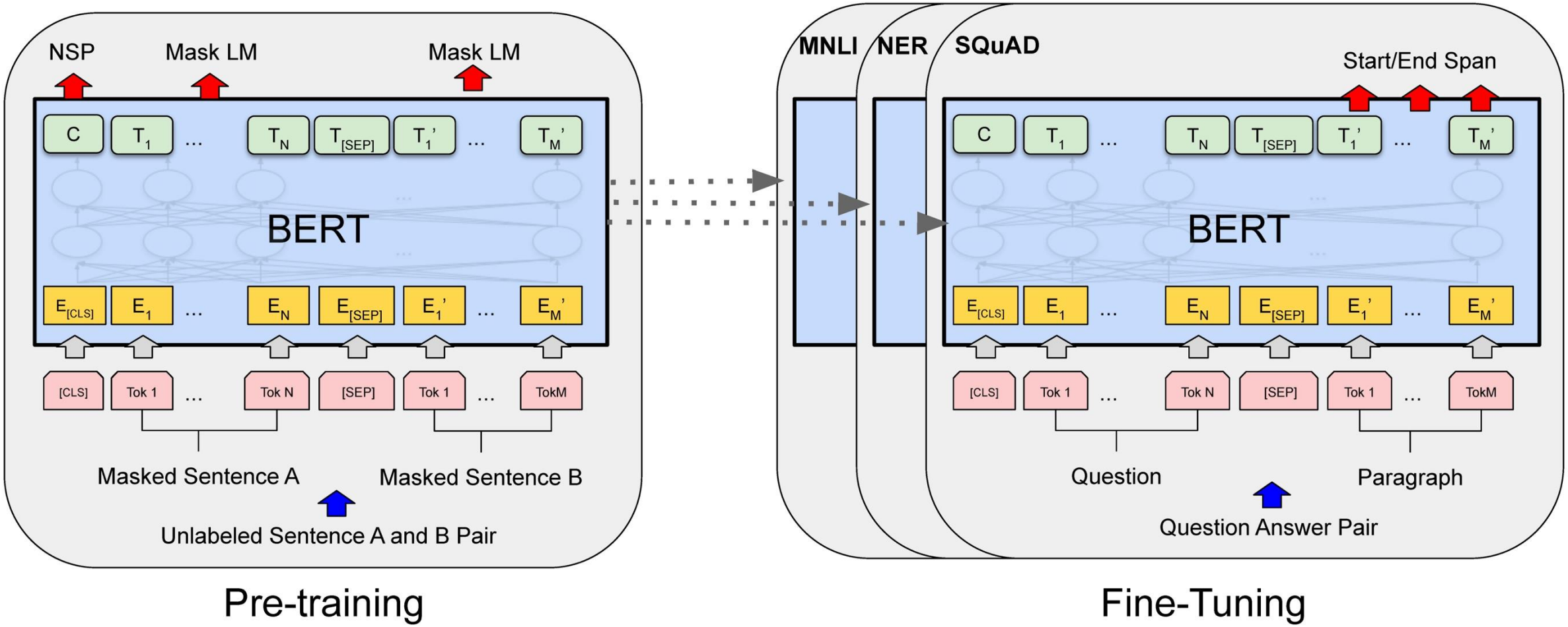

NSP
Mask LM
Mask LM
C
T1
...
TN
T[SEP]
T1'
...
TM'
BERT
E[CLS]
E1
...
EN
E[SEP]
E1'
...
EM'
[CLS]
Tok 1
...
Tok N
[SEP]
Tok 1
...
TokM
Masked Sentence A
Masked Sentence B
Unlabeled Sentence A and B Pair
Pre-training
MNLI
NER
SQuAD
Start/End Span
C
T1
...
TN
T[SEP]
T1'
...
TM'
BERT
E[CLS]
E1
...
EN
E[SEP]
E1'
...
EM'
[CLS]
Tok 1
...
Tok N
[SEP]
Tok 1
...
TokM
Question
Paragraph
Question Answer Pair
Fine-Tuning



- ❑ CLS is a special token identifying part of sequence used for classification tasks
- ❑ At last layer, output C above CLS is used in classification tasks



- Output vector above masked inputs used in a model to predict masked values



- In finetuning, we use the pre-trained model for similar tasks, with paired and unpaired sentences
- "Sentence" is any contiguous unit of language; may be multiple true sentences

GPT vs BERT

- Both use masking
- GPT uses masked attention, yielding an autoregressive model that predicts next word based on the context of preceding k words
- BERT masks out words at random, and it predicts masked words based on vector output from Transformer above masked word
- BERT also constitutes supervised learning based on unlabeled data: next-sentence prediction (NSP)
- GPT does supervised learning from scratch at fine-tuning, based on an appended linear layer
- BERT learns how to distinguish by a special classification token, pre-trained with NSP

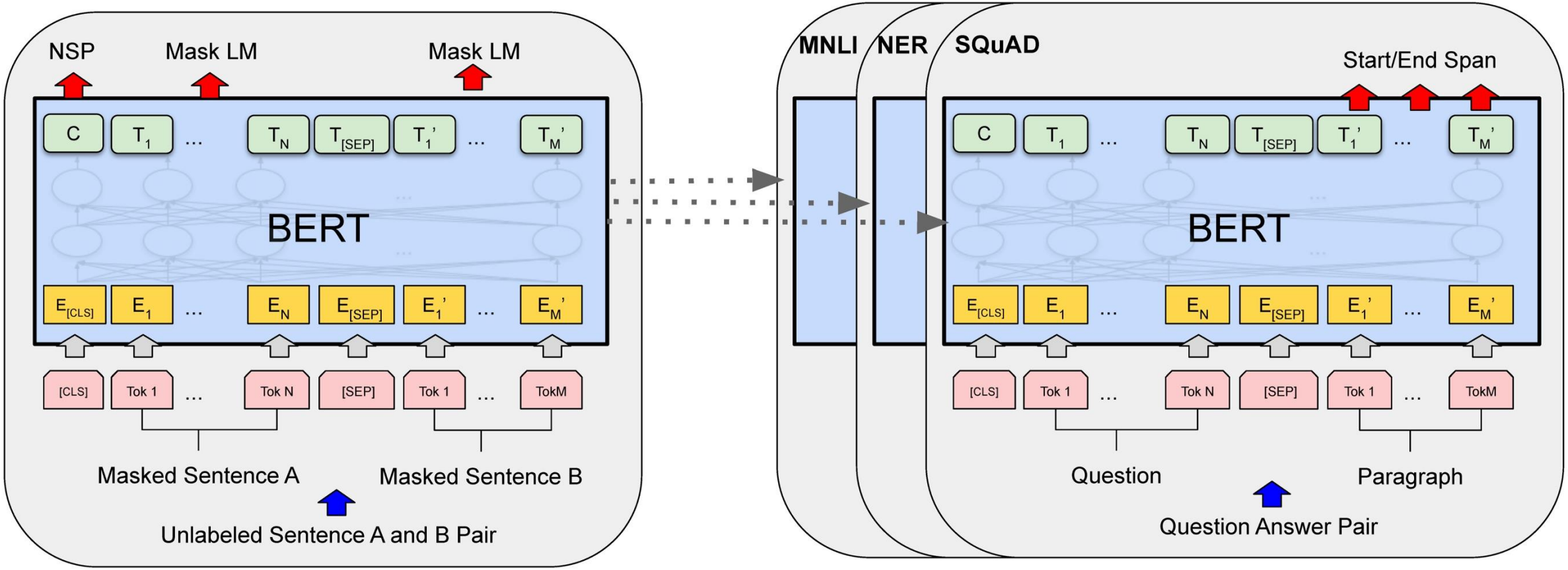
GPT vs BERT

- GPT is designed for next word predictions, so well suited for generative AI, and text synthesis

- BERT more appropriate for constrained tasks, such as multiple-choice questions, or finding answers in given text

- BERT can also be used for tasks like sentiment analysis

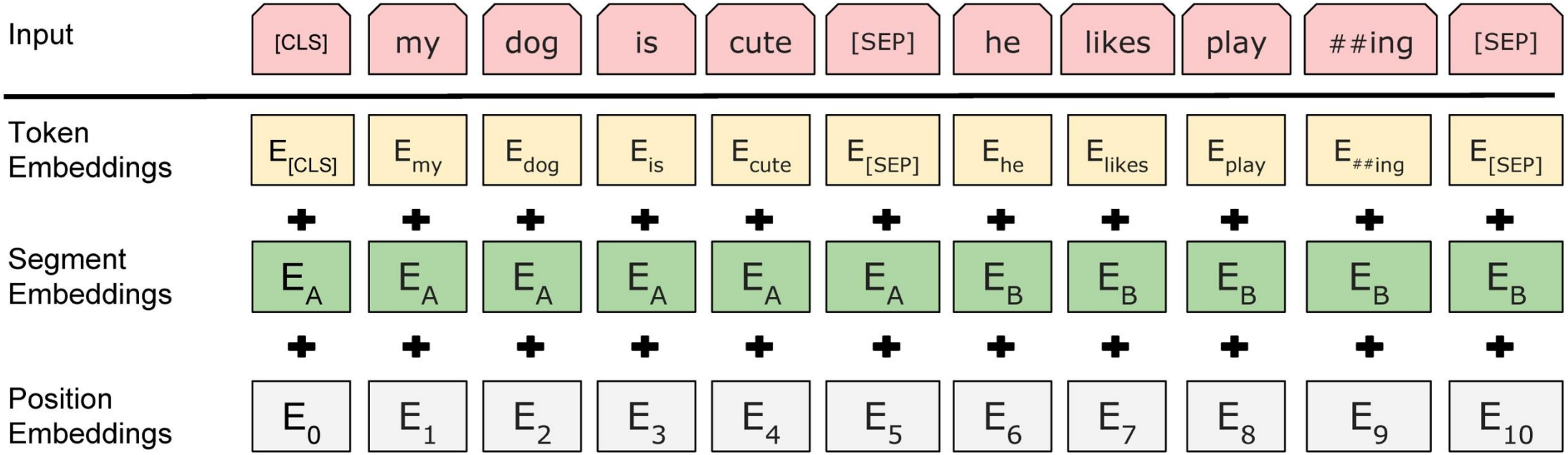
Technical Details of BERT



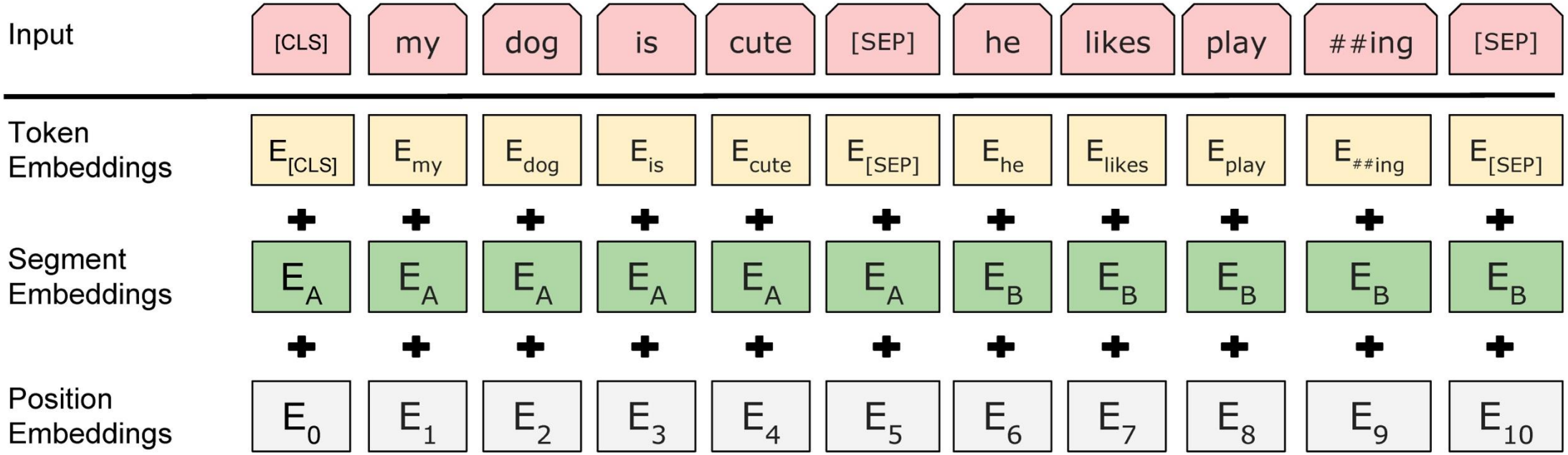
Pre-training

Fine-Tuning

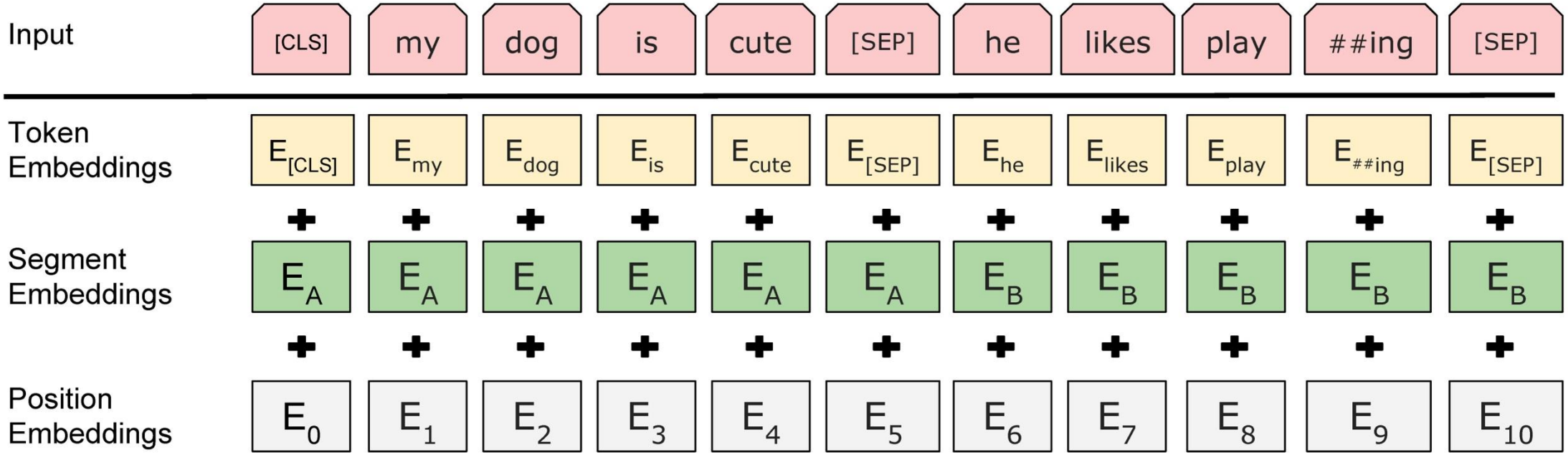
Embedding the Input Sequence



Token Embeddings for Masked Tokens

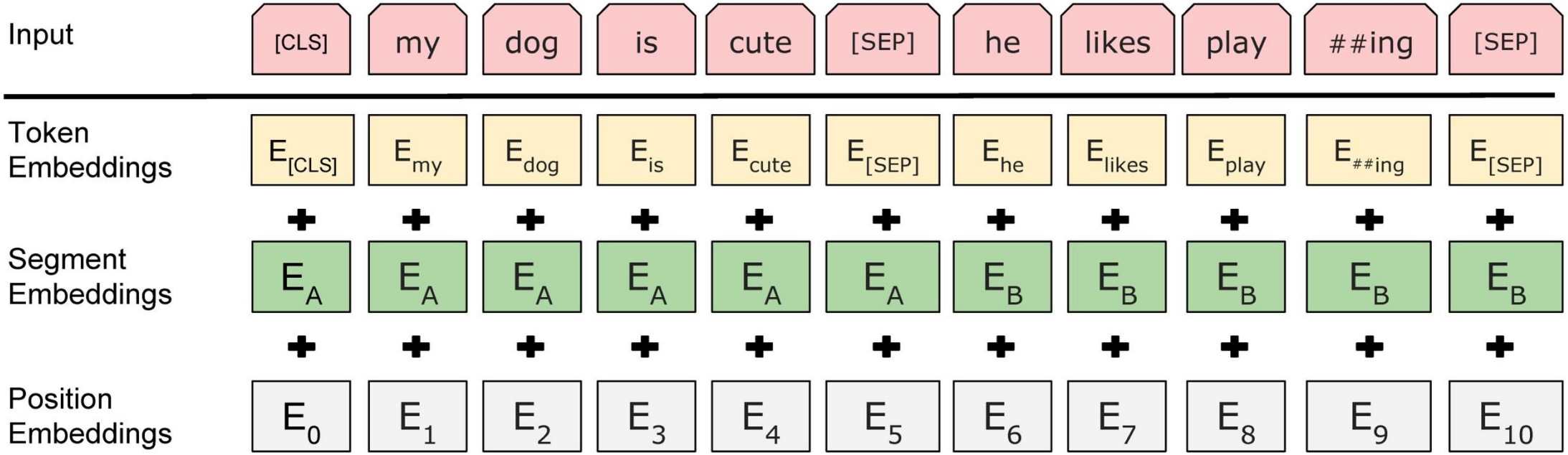


Token Embeddings for Masked Tokens



- 15% of input tokens masked

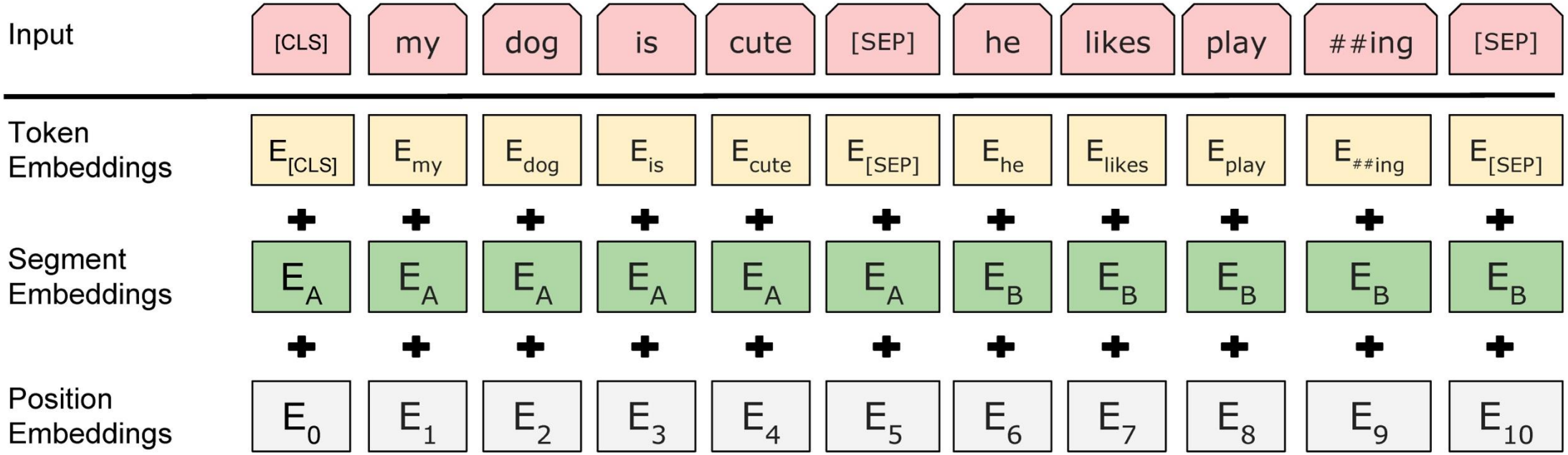
Token Embeddings for Masked Tokens



- Of these, for 80% a special [MASK] token used (learned)

- 15% of input tokens masked

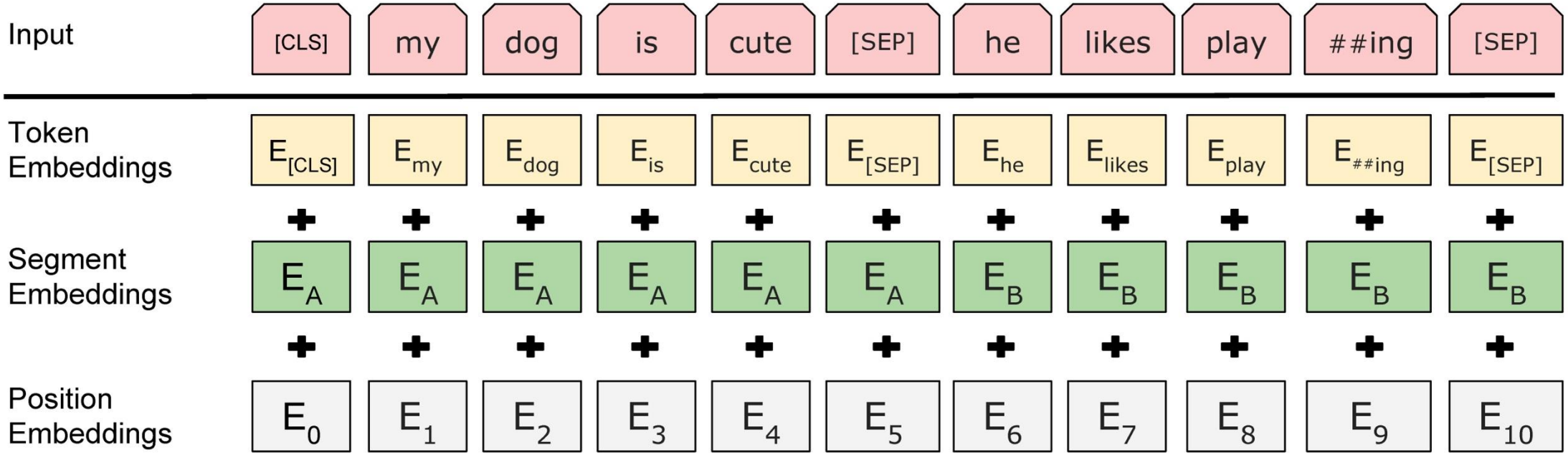
Token Embeddings for Masked Tokens



- 15% of input tokens masked

- Of these, for 80% a special [MASK] token used (learned)
- 10% of time put a random word at position of mask
- 10% put the original word in

Token Embeddings for Masked Tokens



- 15% of input tokens masked

- Of these, for 80% a special [MASK] token used (learned)
- 10% of time put a random word at position of mask
- 10% put the original word in

Use output above masked word to predict original word

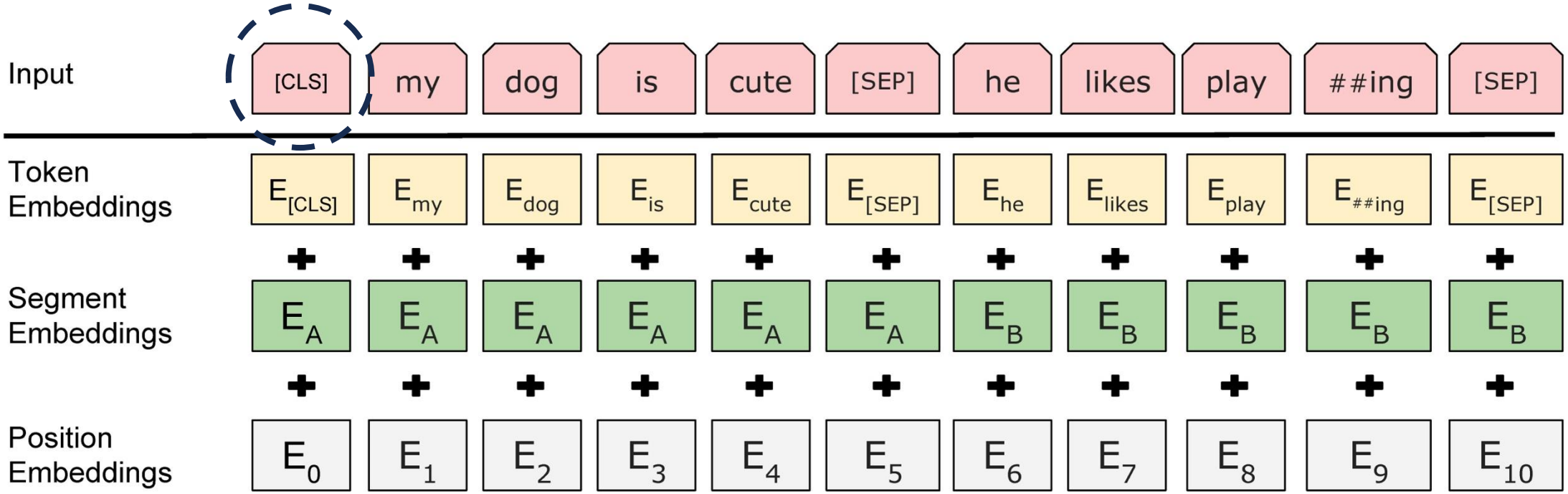
Learn to Classify When Pretraining

- BERT learns how to predict masked words based on surrounding words (not autoregressive)
- Bidirectional attention
- It also learns to classify, based on unlabeled data
- Next-sentence prediction (NSP)
- Effectively learns features of *sentences* that are predictive of other things
- Requires a learning of structure and meaning of language, along with prediction of masked words which also manifests an understanding of language

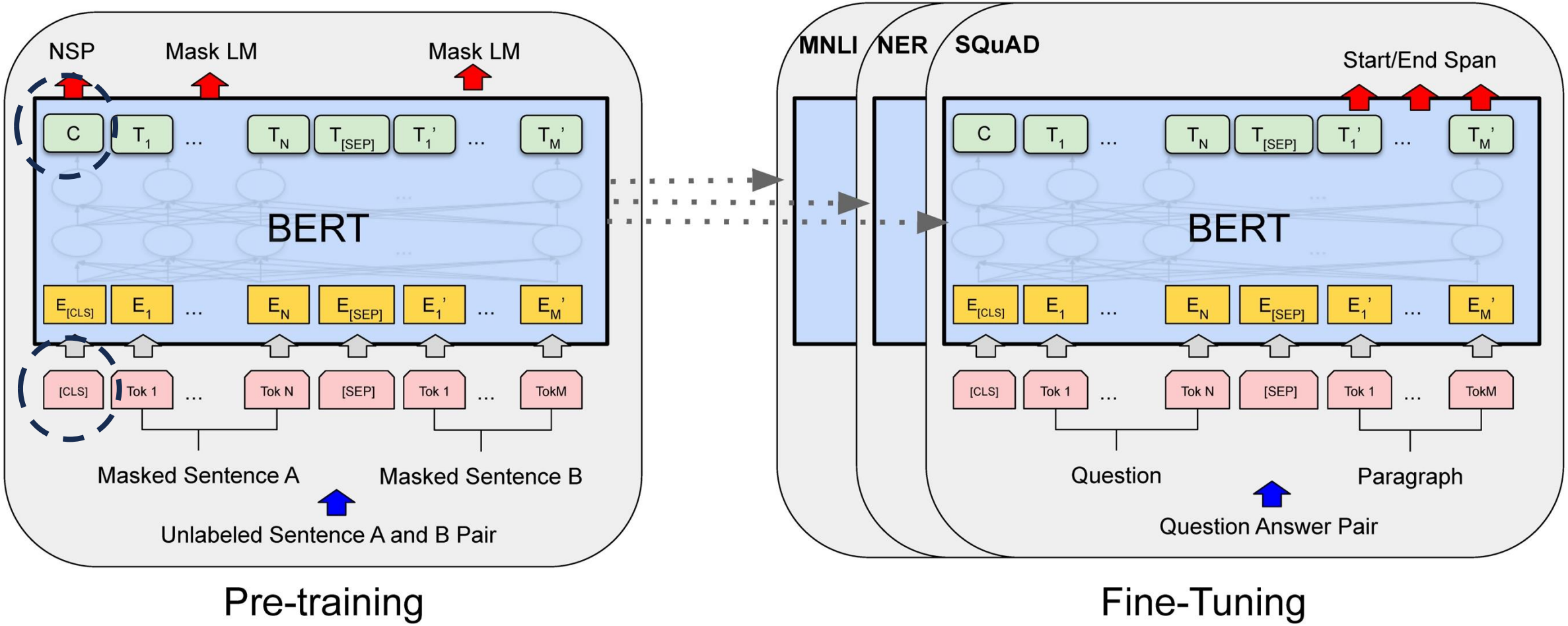
Learn to Classify When Pretraining

- The special token and resulting features used for pretraining are then fine-tuned for down-stream predictive tasks
- Multiple-choice questions
- Find answer to question within given text
- Sentiment analysis
- Match similar texts, in retrieval of documents from a database

Special Classification Token (learned embedding)

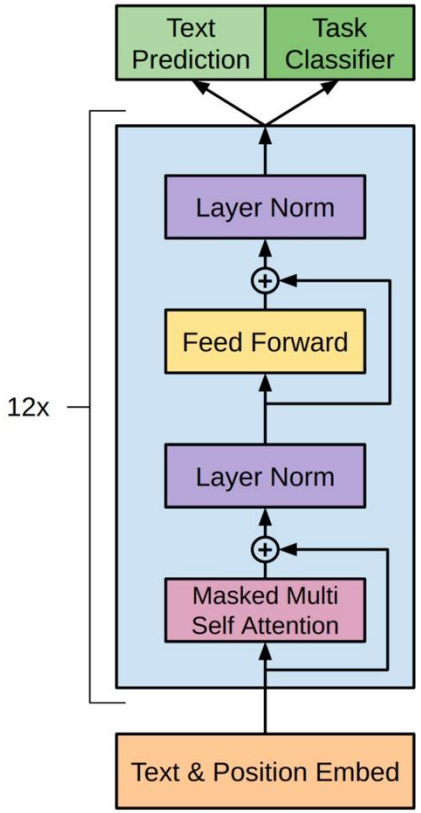


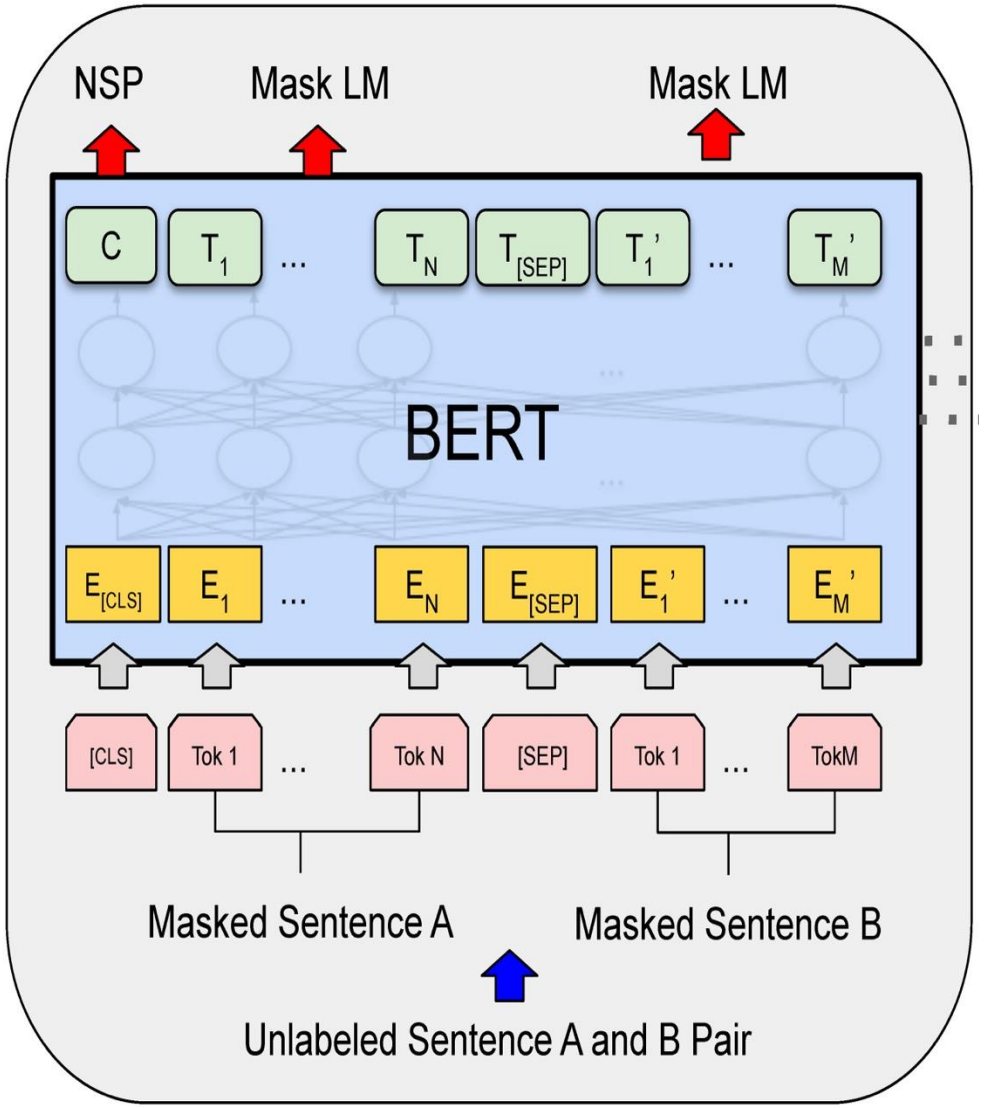
Learning to Classify with BERT



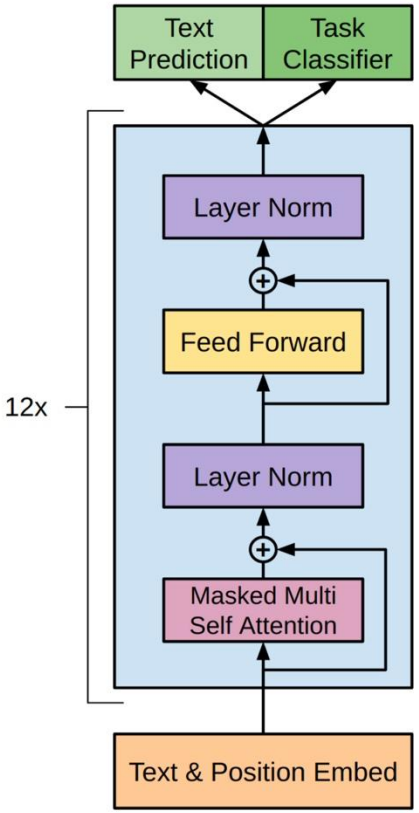
Output vector C above [CLS] is sent into a linear classifier

Key Attribute of all LLMs

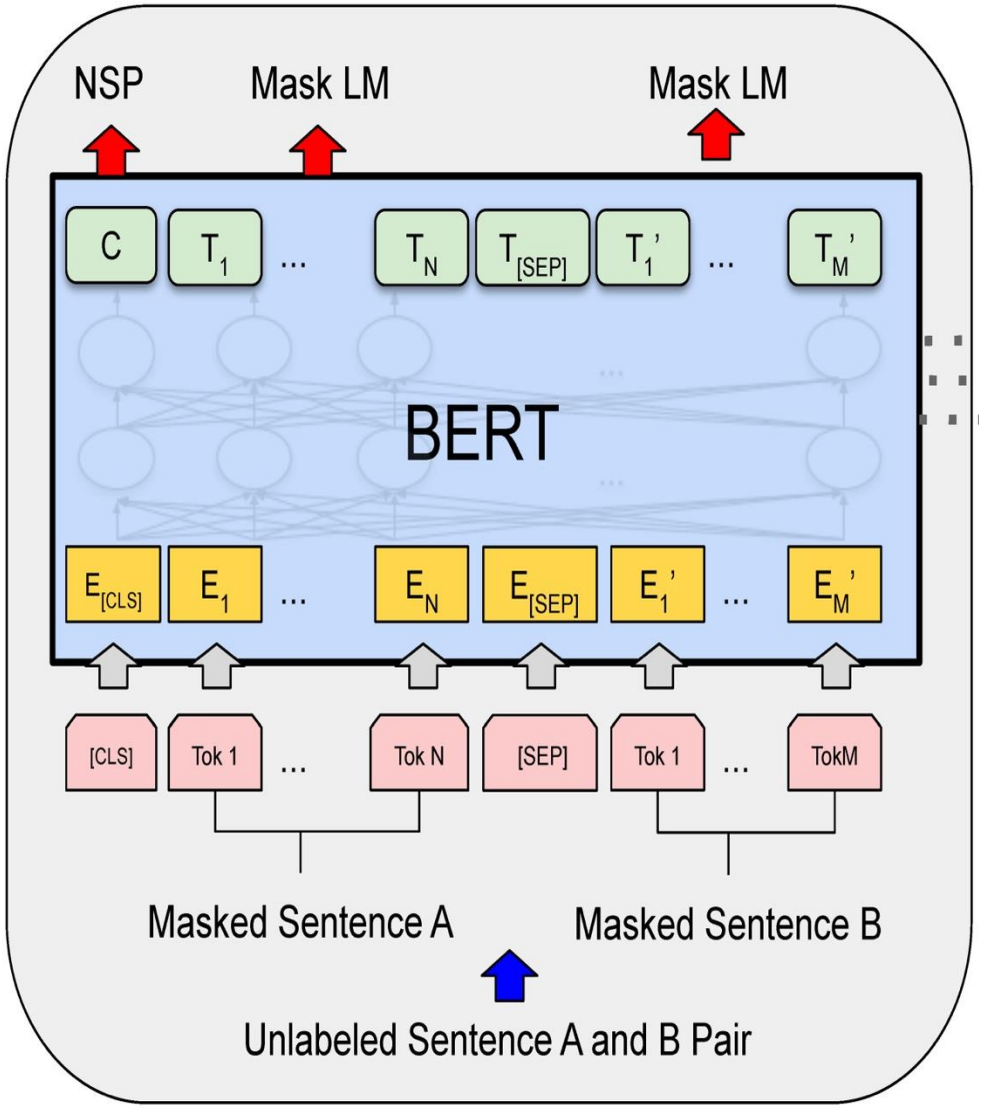




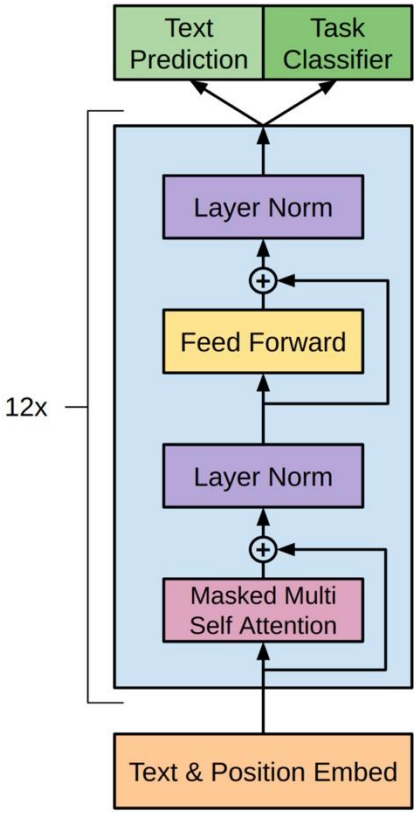
Key Attribute of all LLMs

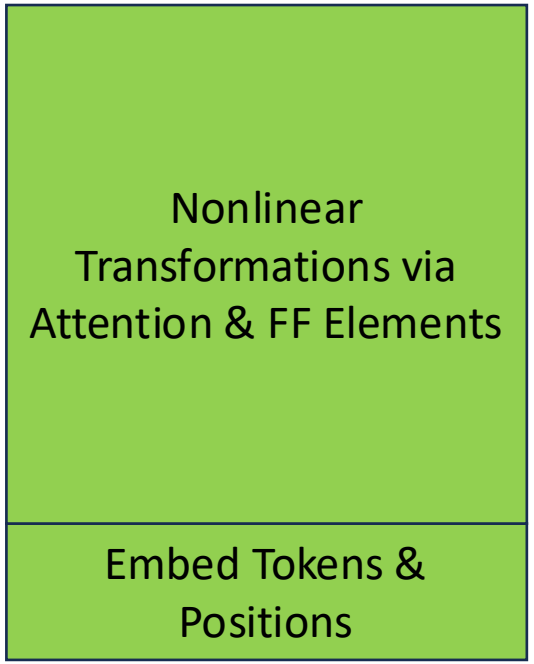


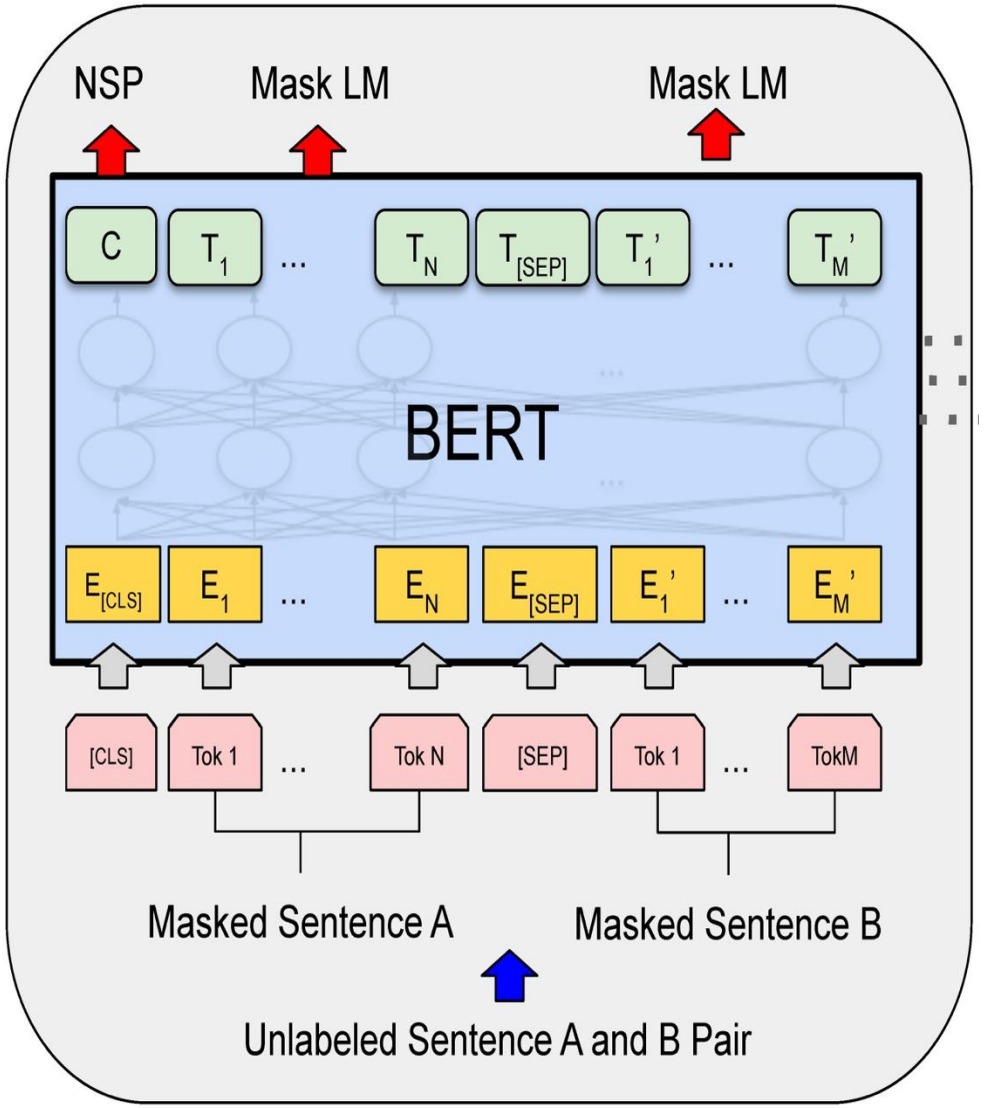
Embed Tokens & Positions



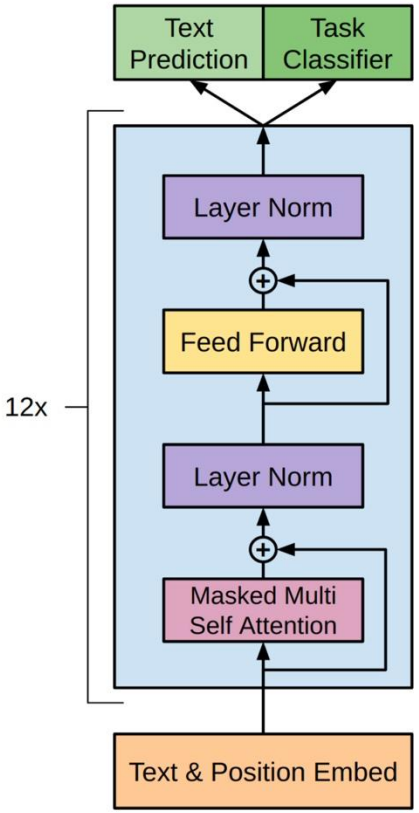
Key Attribute of all LLMs

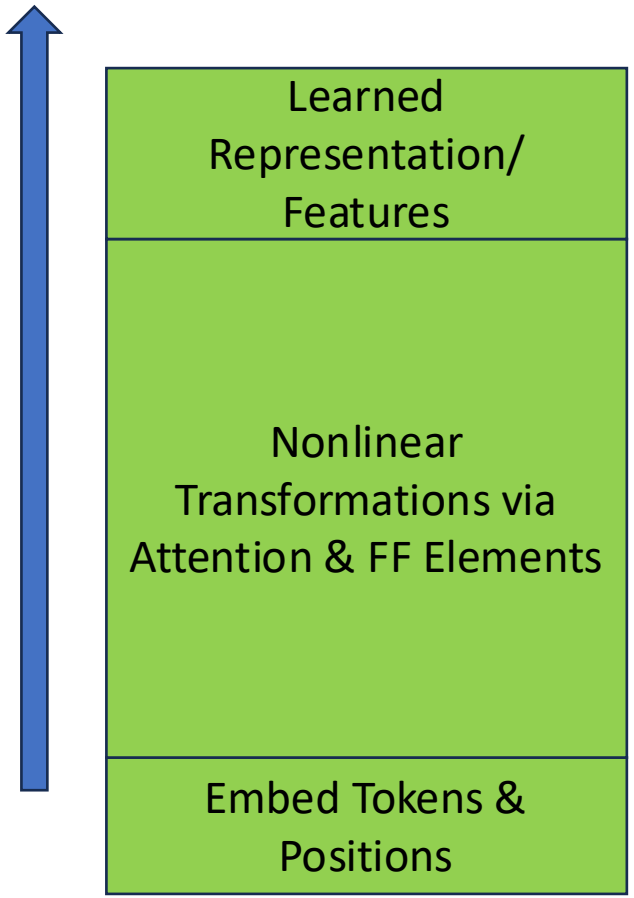


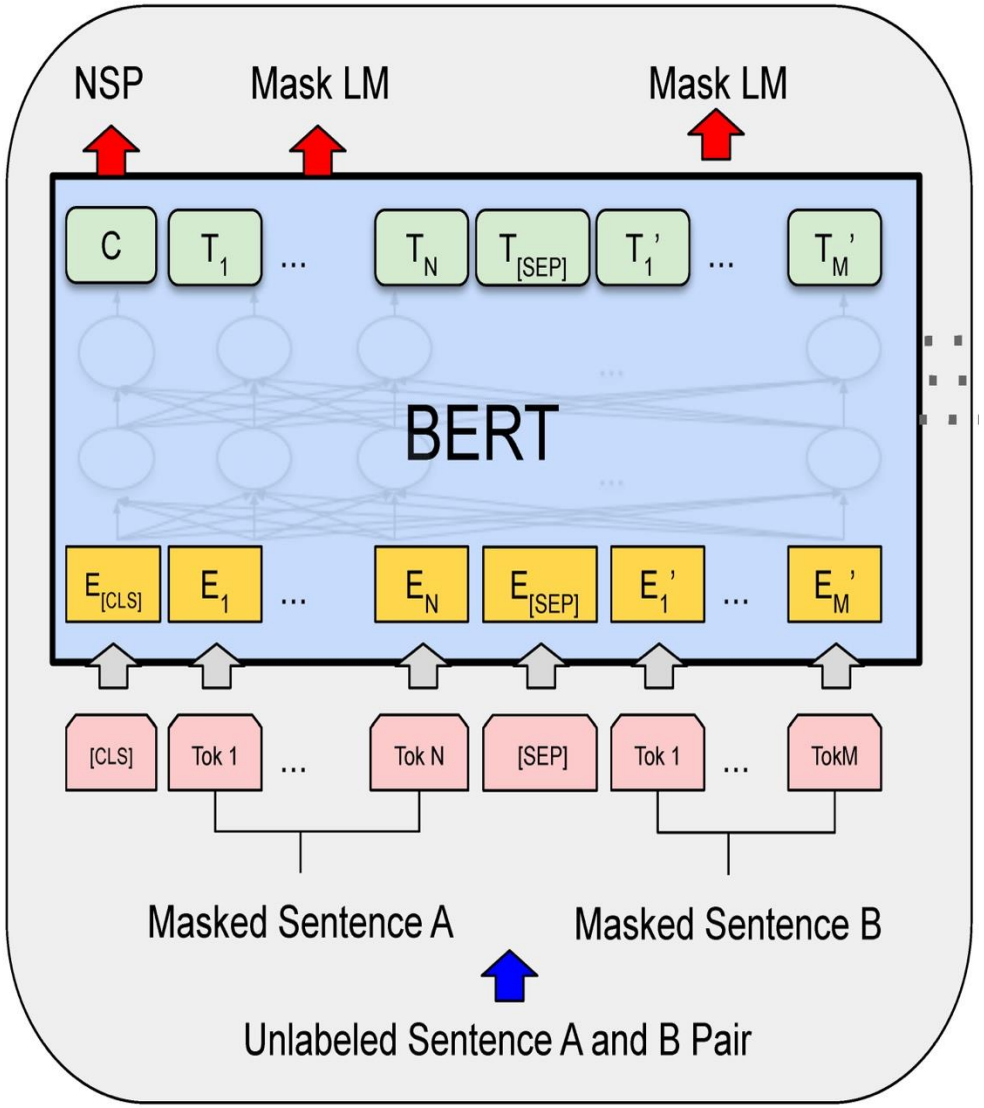
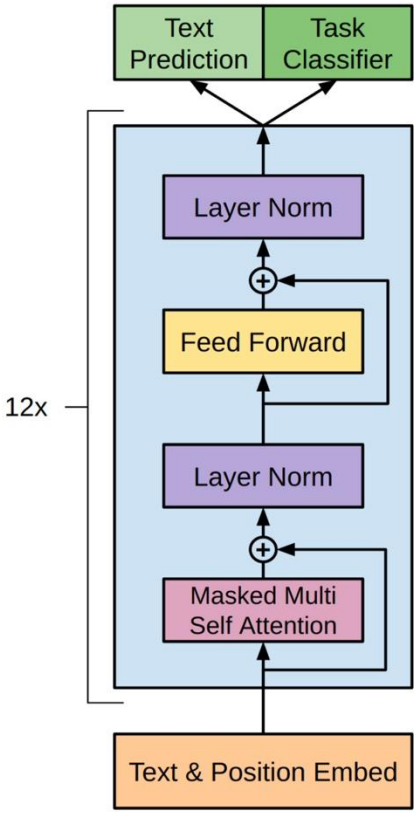
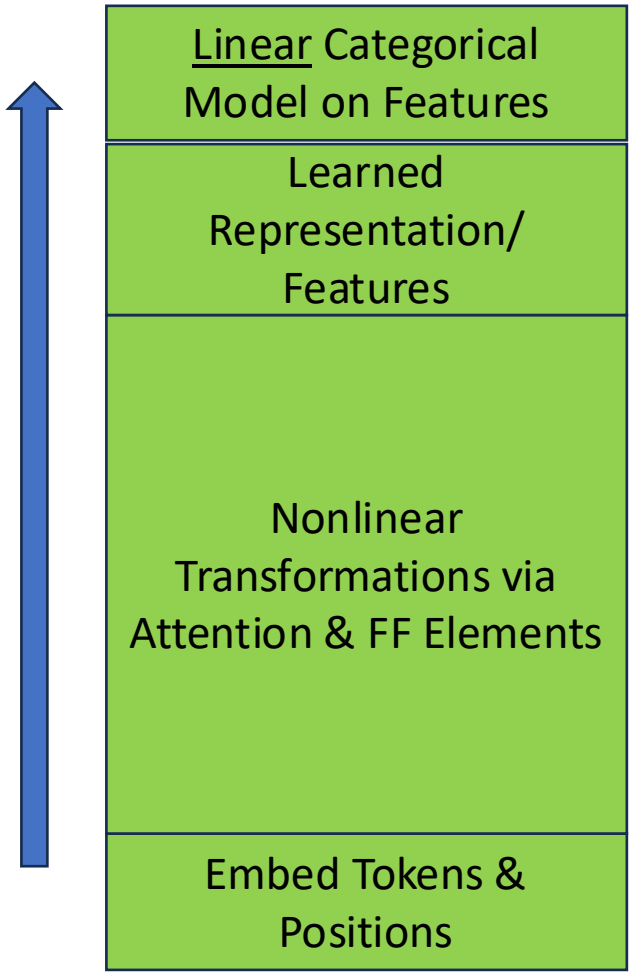
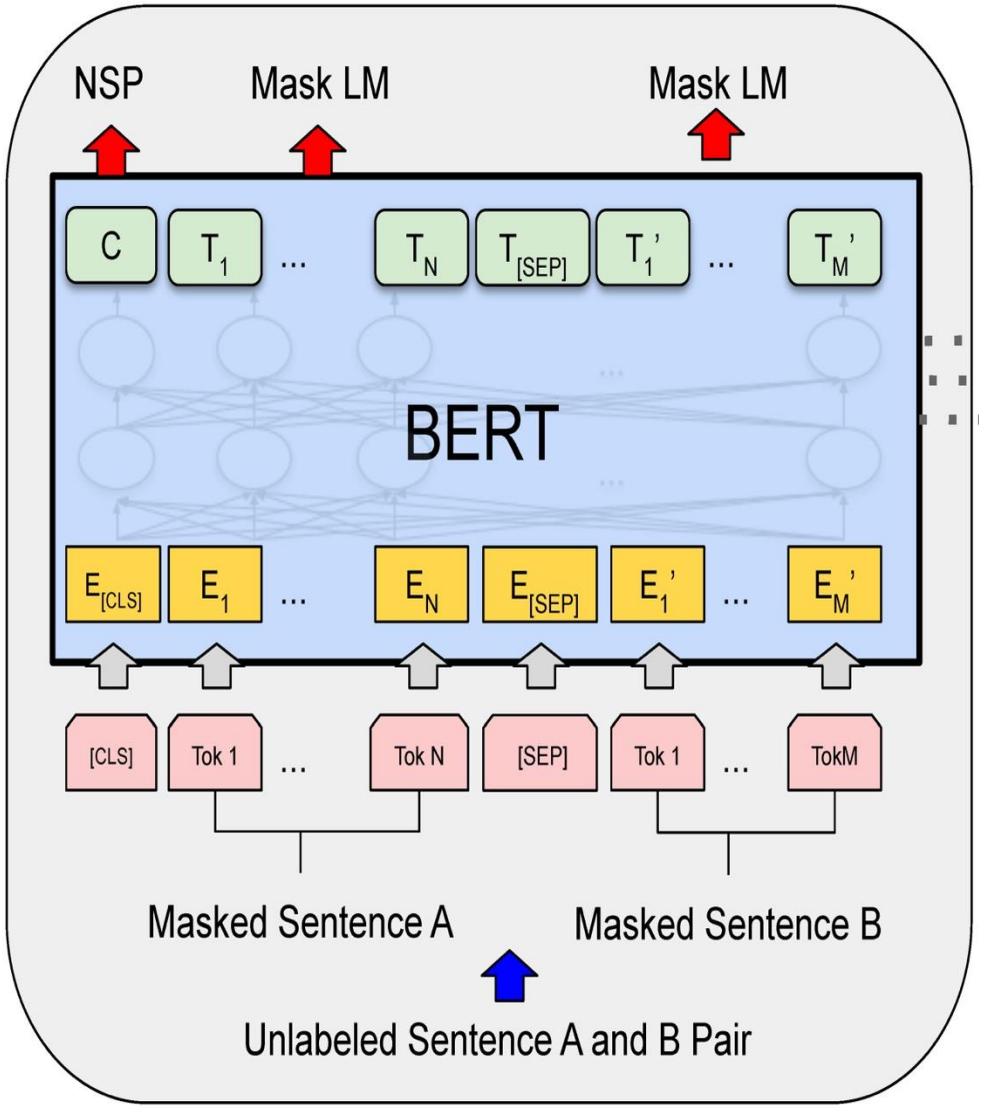




Key Attribute of all LLMs







Key Attribute of all LLMs